

To adapt or not? That is the Ski question.

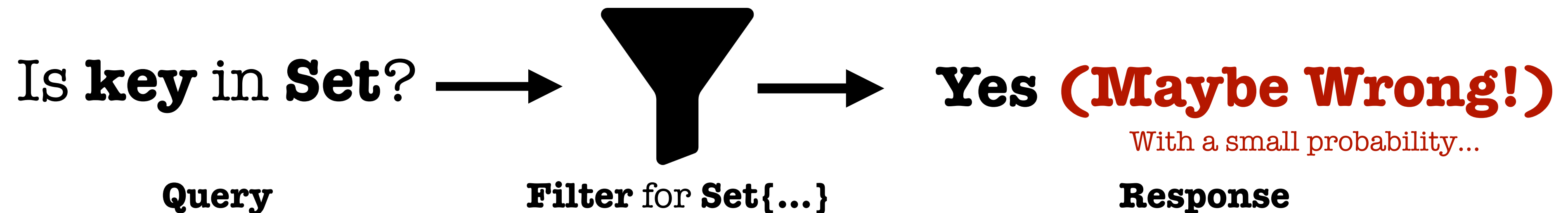
**Online Adaptive Filters using Ski-Rental Algorithms
for Robust Performance**

NEDB Day, Jan 2026
Yuvaraj Chesetti, Prashant Pandey
Northeastern University

***Under submission**

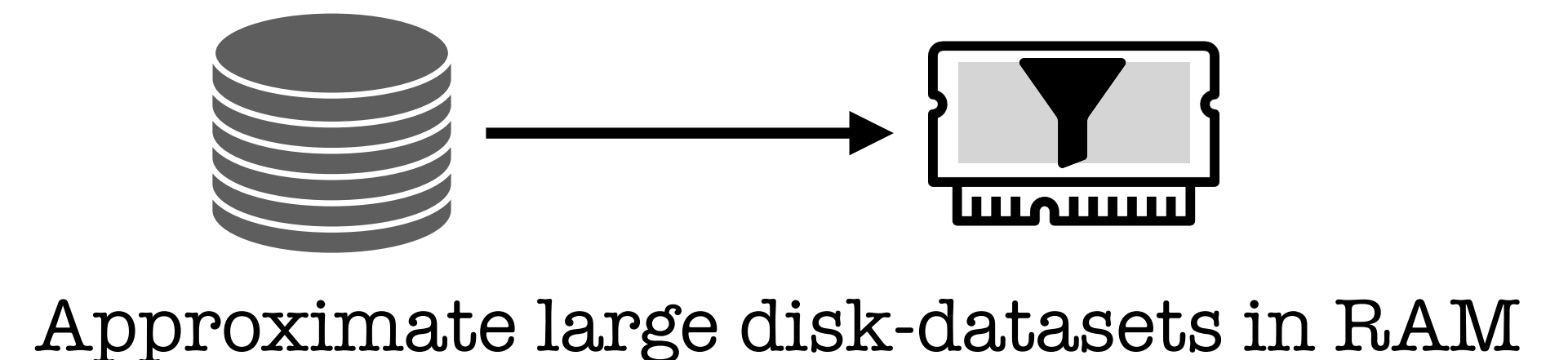
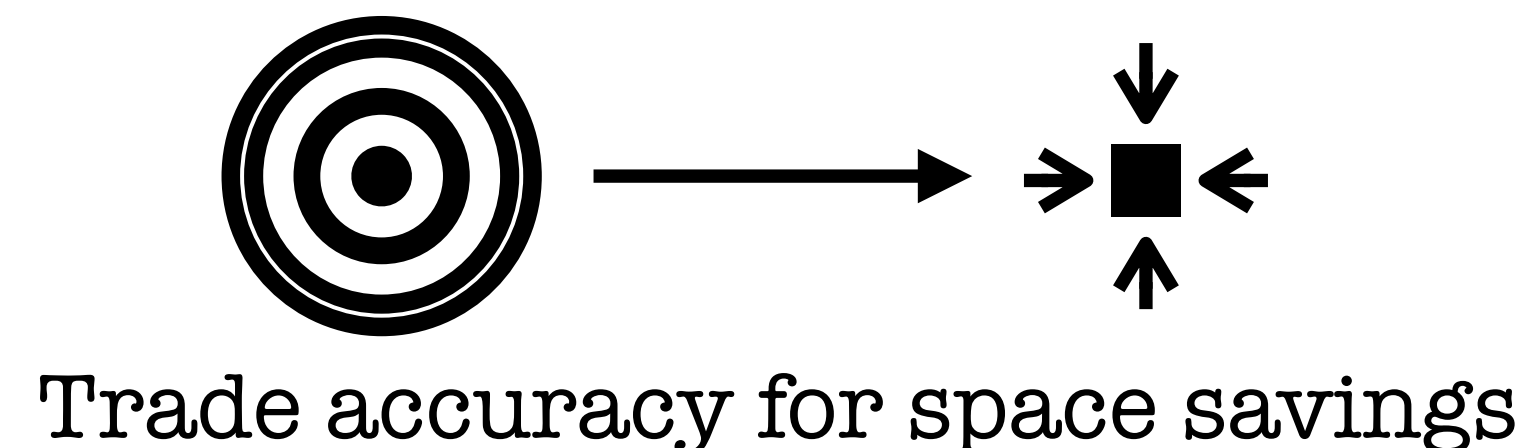
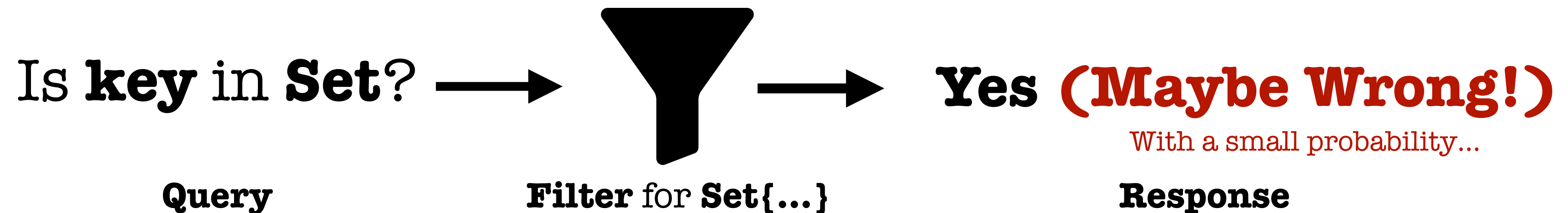
What is an “adaptive” filter?

A **filter** answers **membership queries** for a **set** with
no false negatives and **potential false positives**



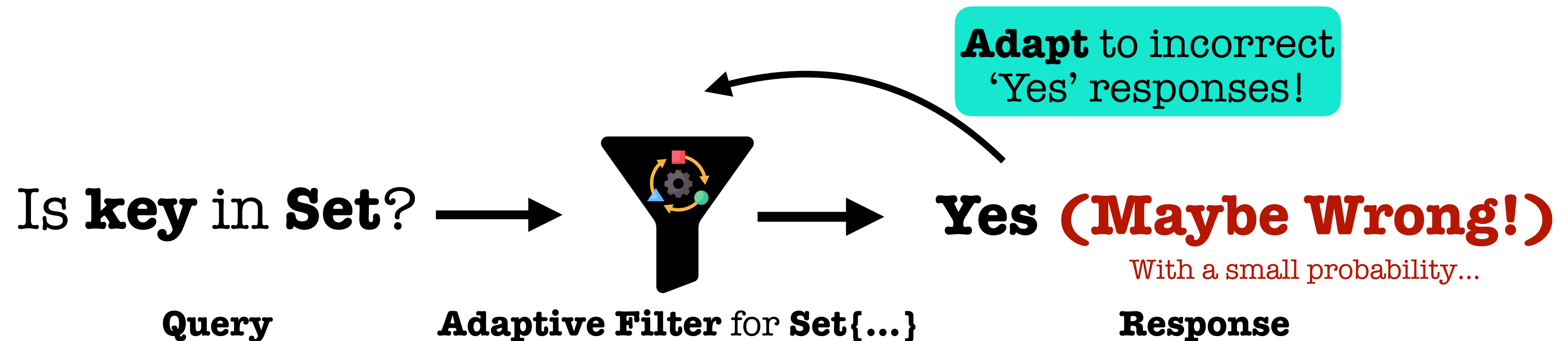
What is an “adaptive” filter?

A **filter** answers **membership queries** for a **set** with
no false negatives and **potential false positives**

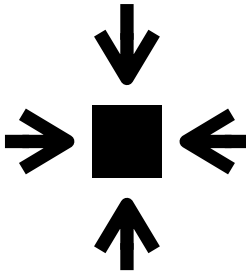


What is an “adaptive” filter?

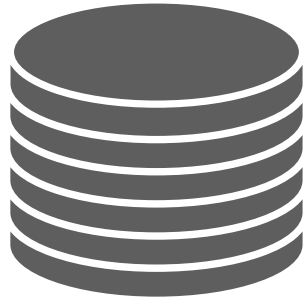
An ‘**adaptive**’ filter answers **membership queries** for a **set** with
no false negatives and **potential false positives**,
but **learns to avoid repeating errors!**



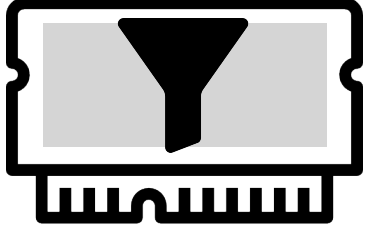
→



Trade accuracy for space savings

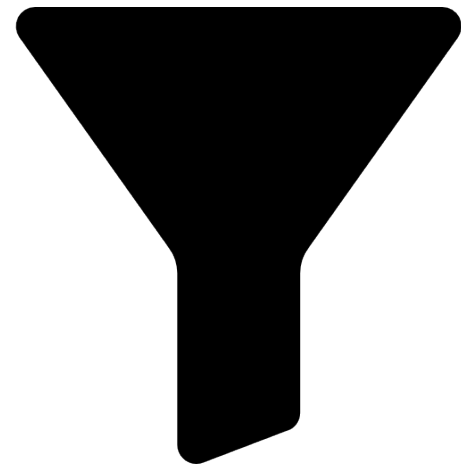


→

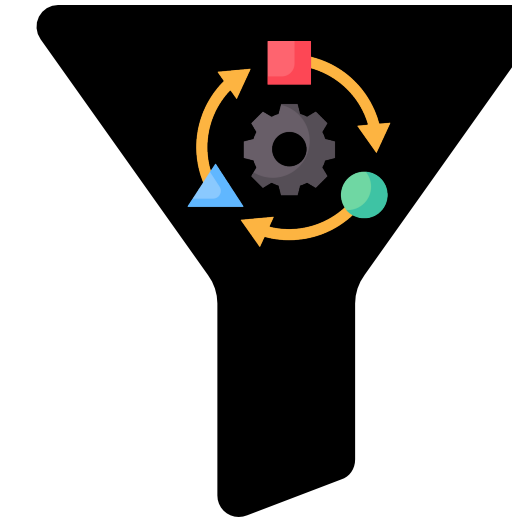


Approximate large disk-datasets in RAM

Traditional Filter



Adaptive Filter



Set: { A, E, I, O, U }

Is **F** in set? YES (**False positive**)

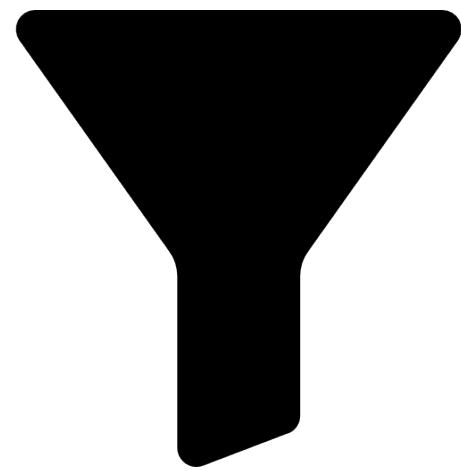


Is **F** in set? YES (**False positive**)

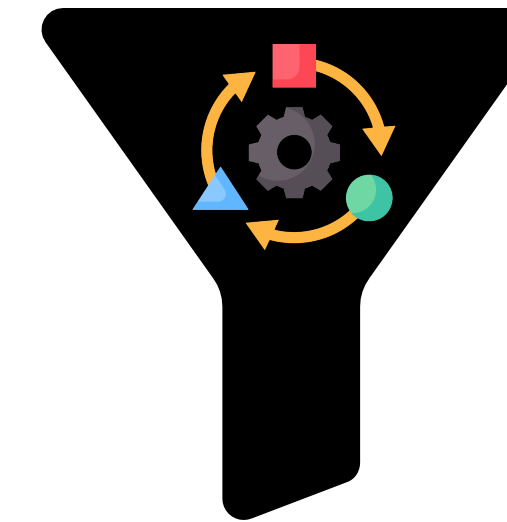
Adapting...



Traditional Filter



Adaptive Filter



Set: { A, E, I, O, U }

Is **F** in set? YES (**False positive**)

Is **F** in set? YES (**False positive**)

Is **F** in set? YES (**False positive**)



Is **F** in set? YES (**False positive**)

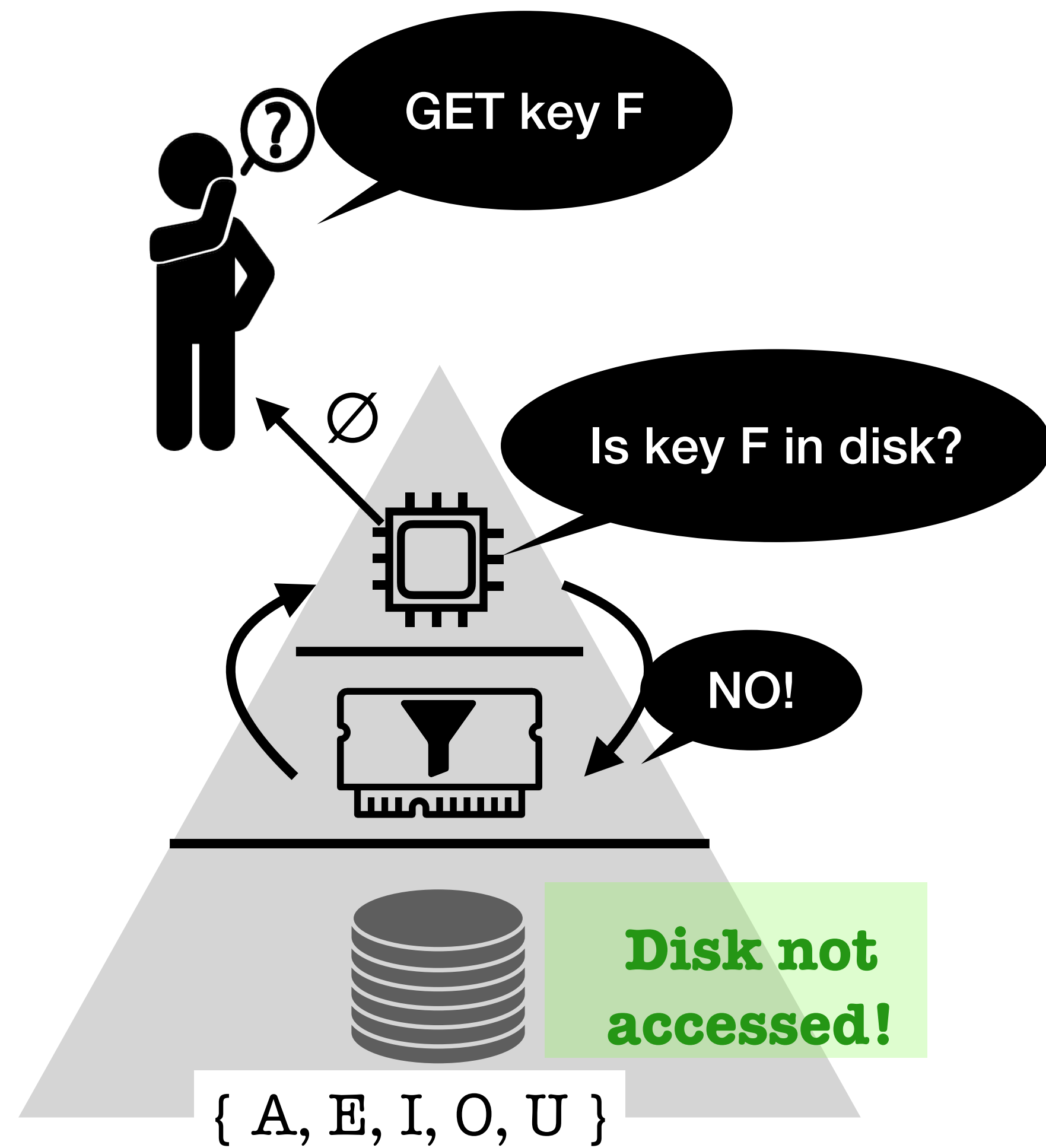
Is **F** in set? NO (**True negative**)

Is **F** in set? NO (**True negative**)

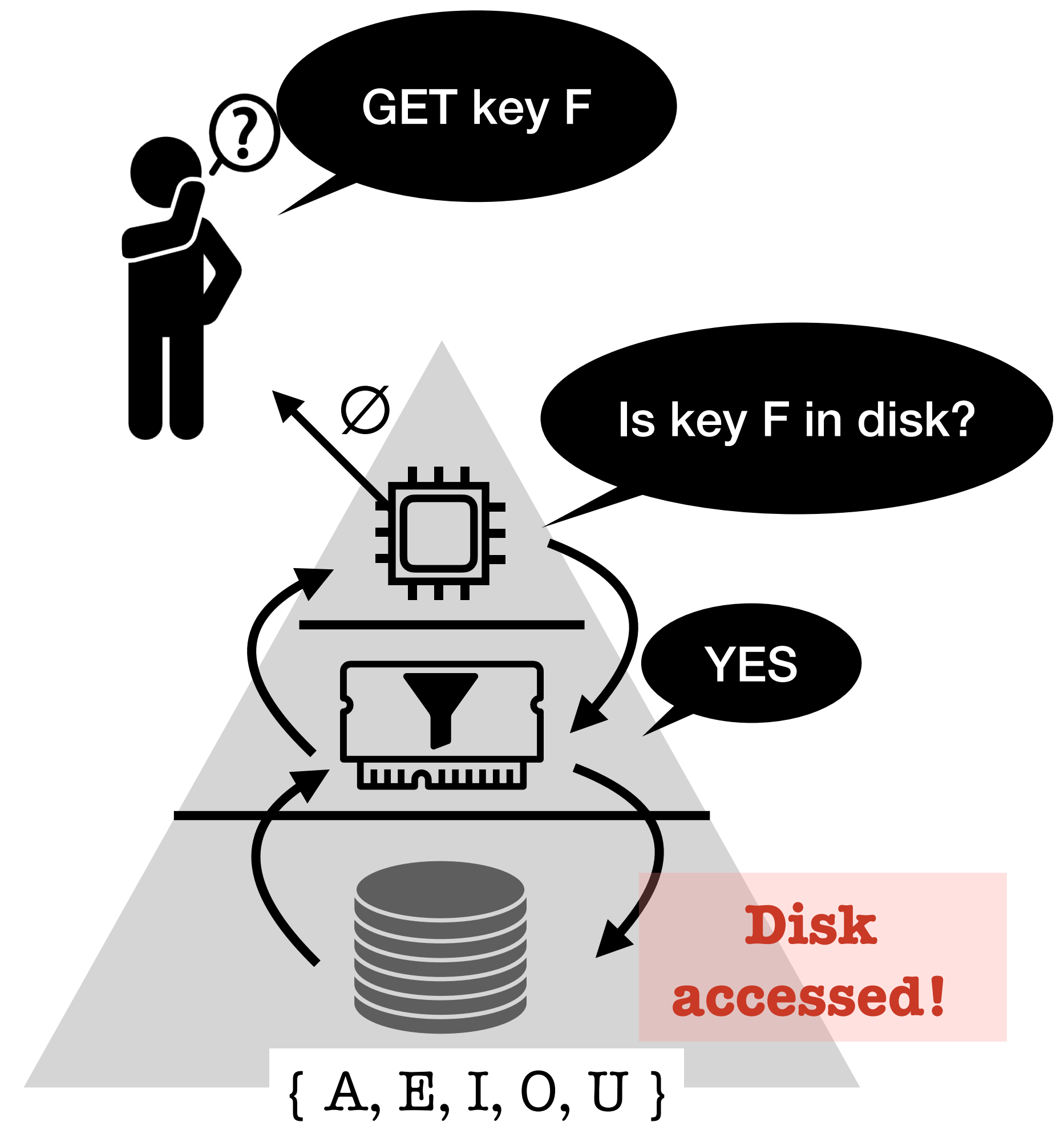
Adapting...



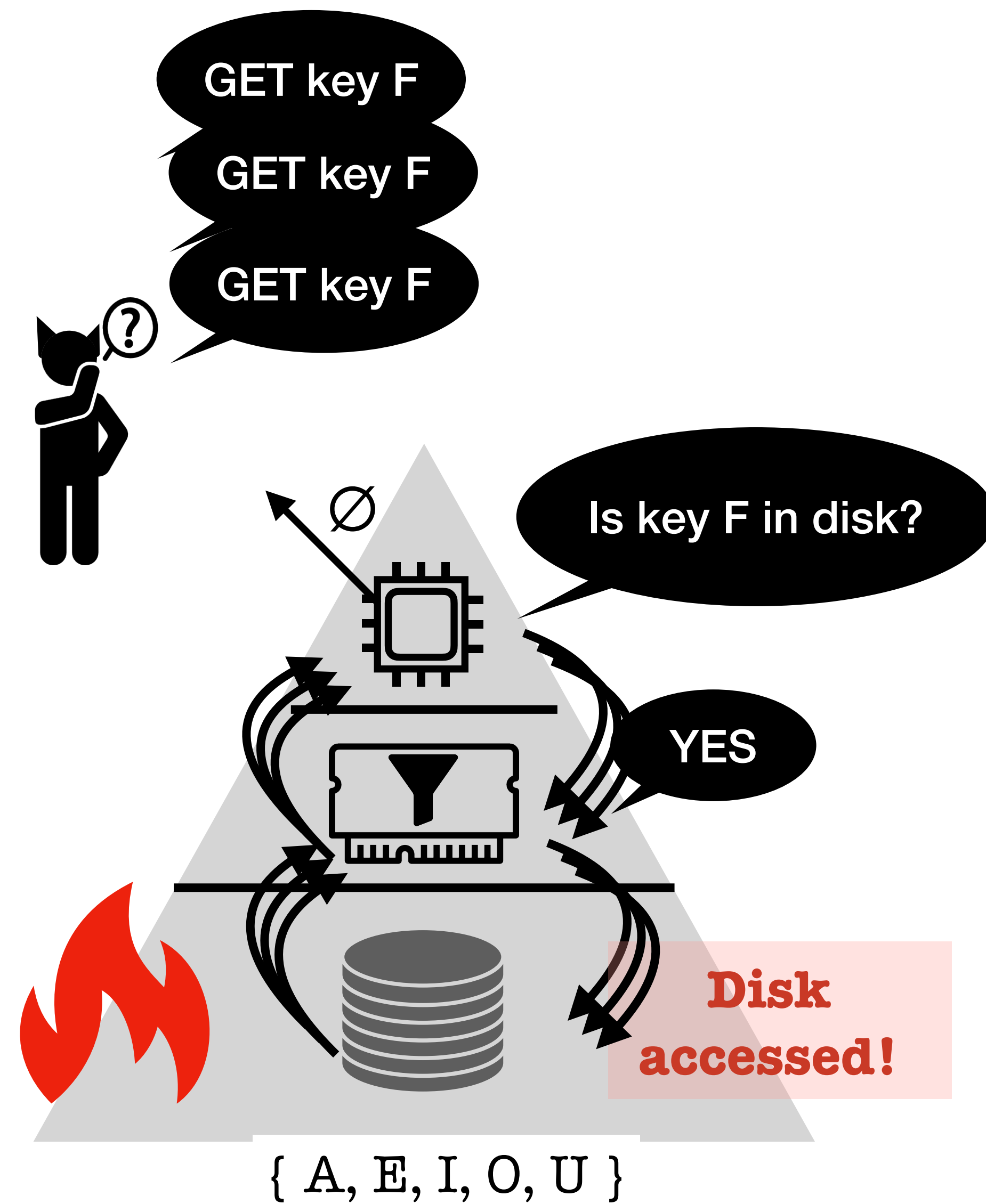
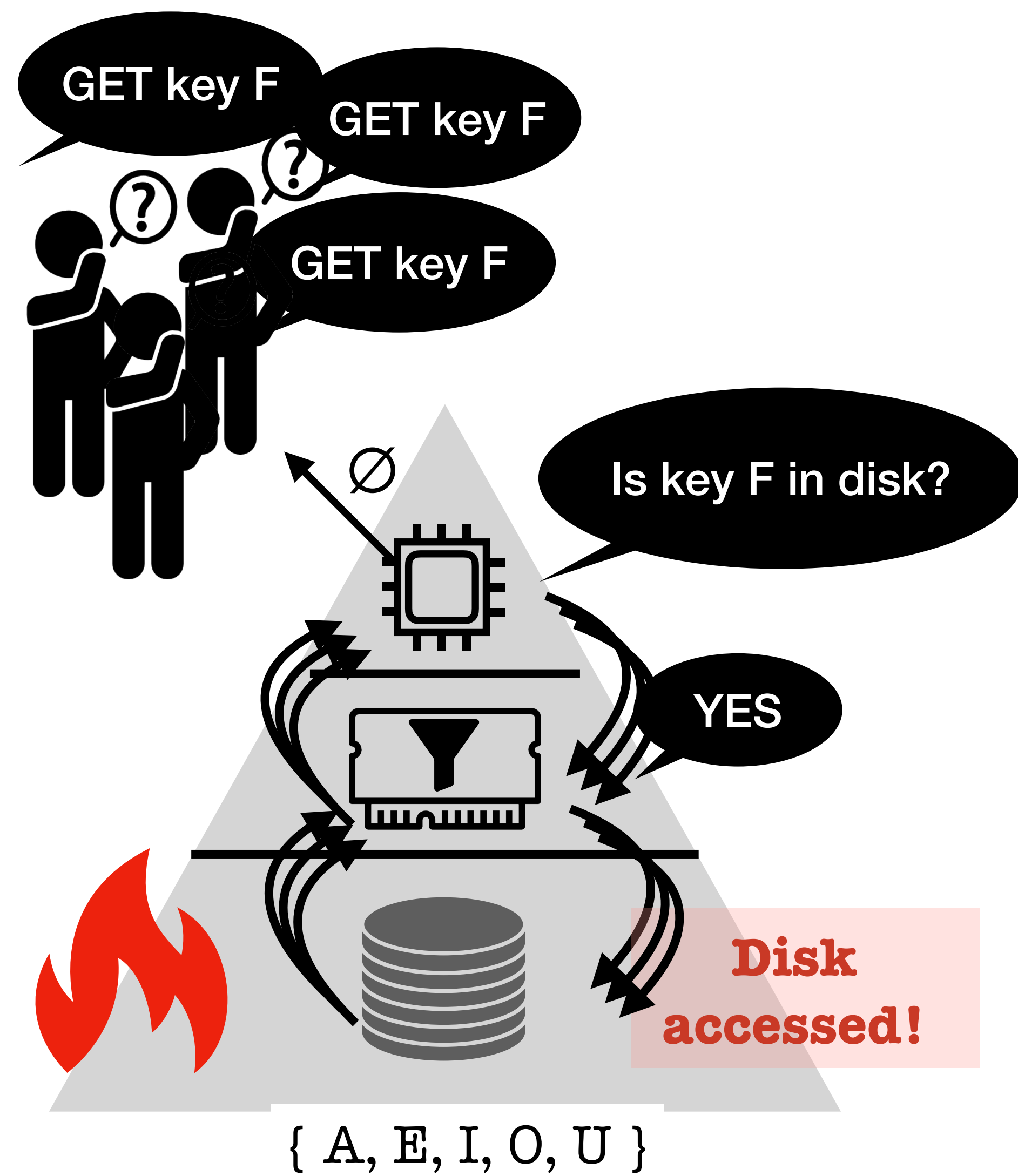
Adaptive filters **avoid error repetitions!**



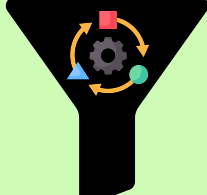
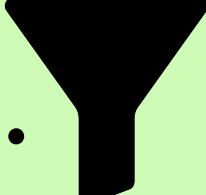
Skip disk I/O if filter says NO

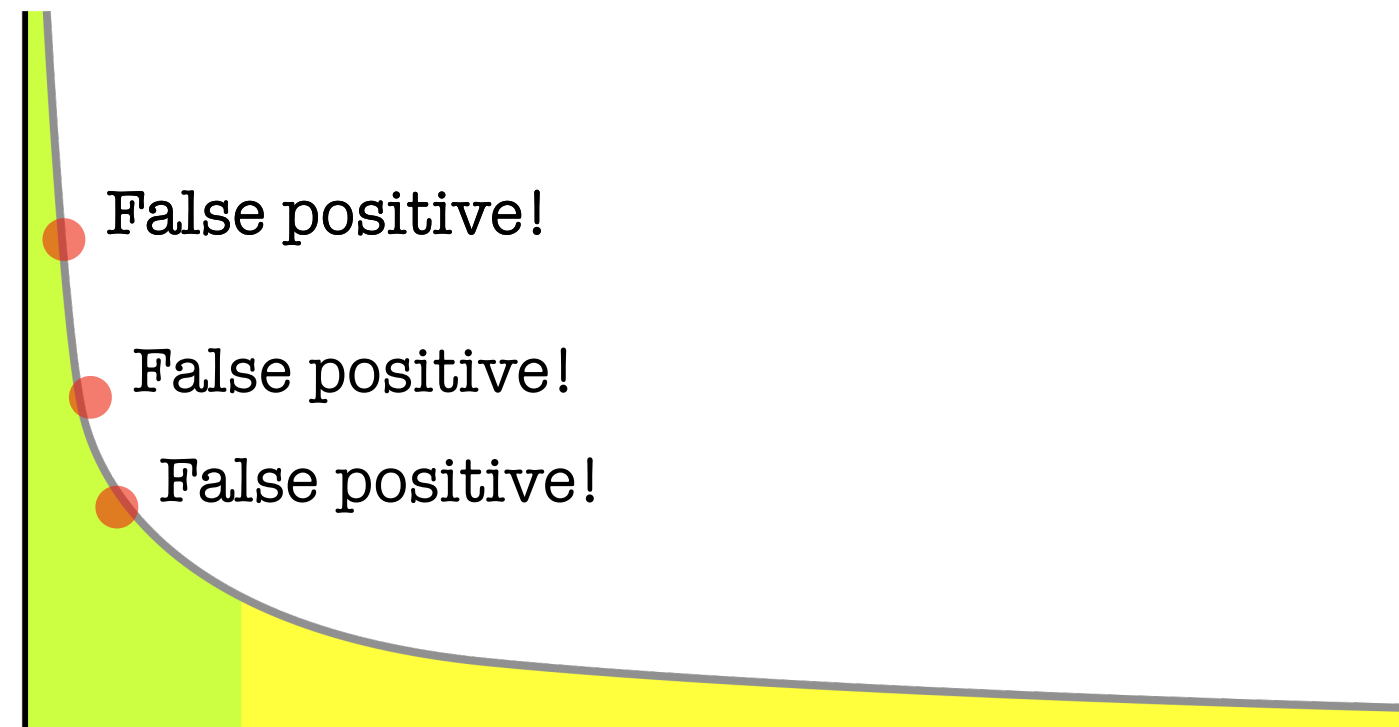


False positives result in Disk I/O



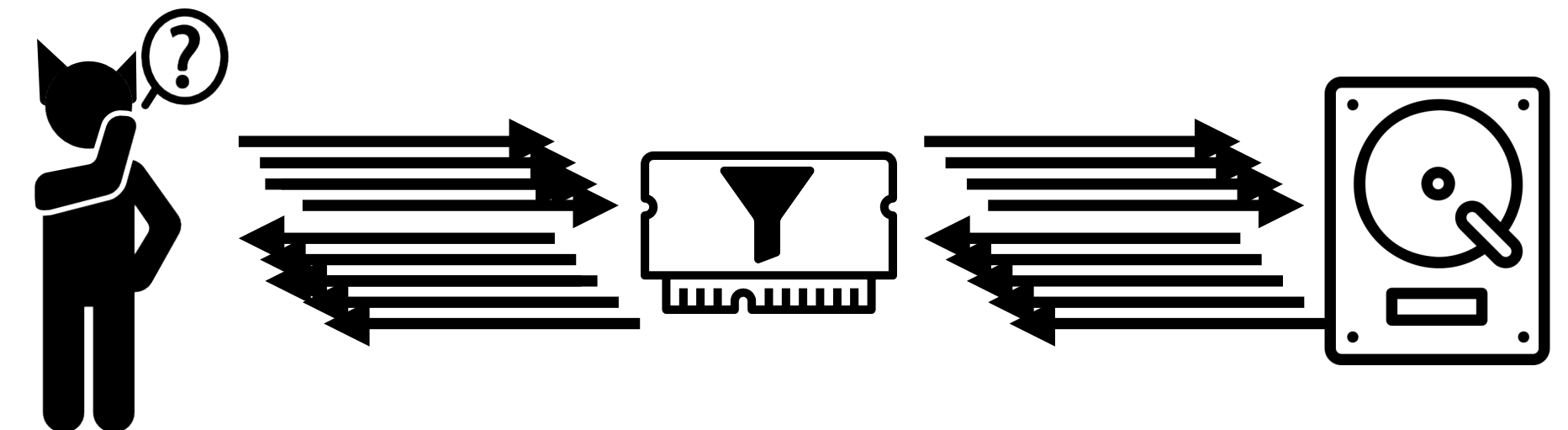
Adaptive filters improve system performance when false positive repeat

 Vs.  : **1.3 - 12x** (Throughput)



Skewed workloads

Popular queries may happen to be a false positive

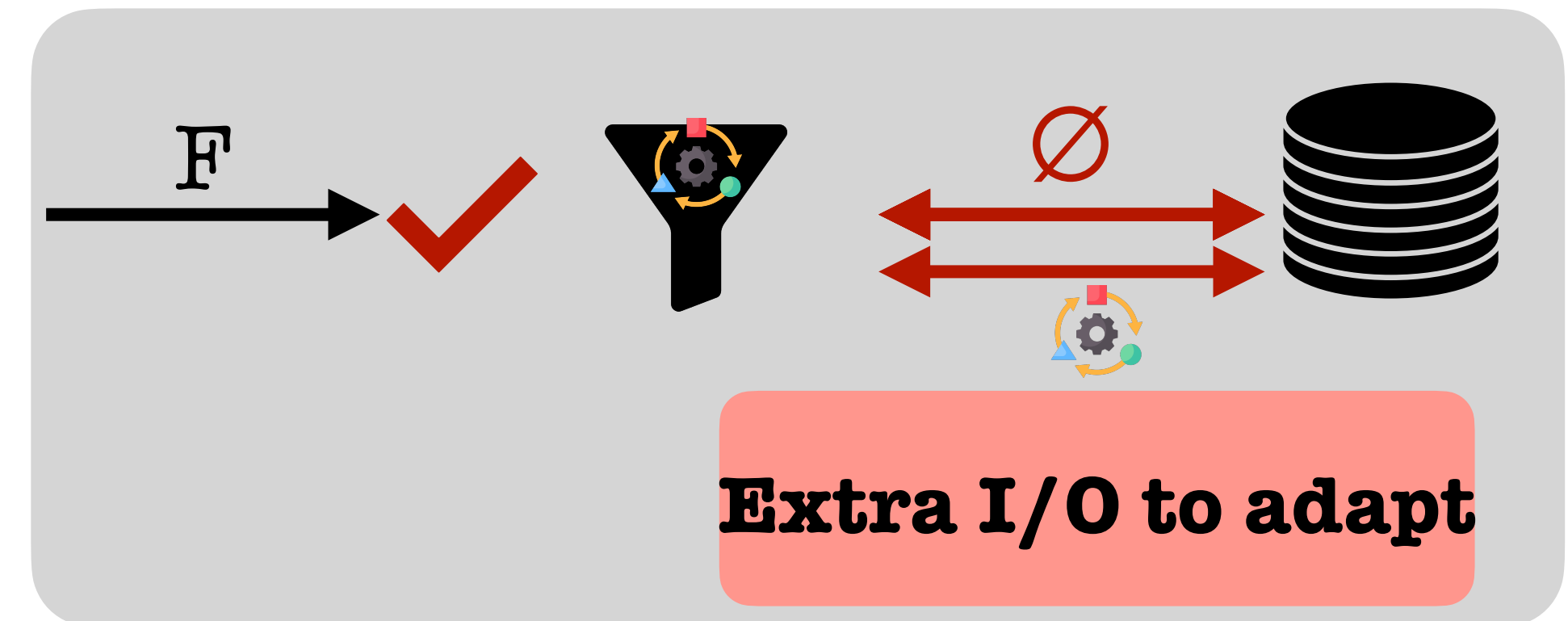
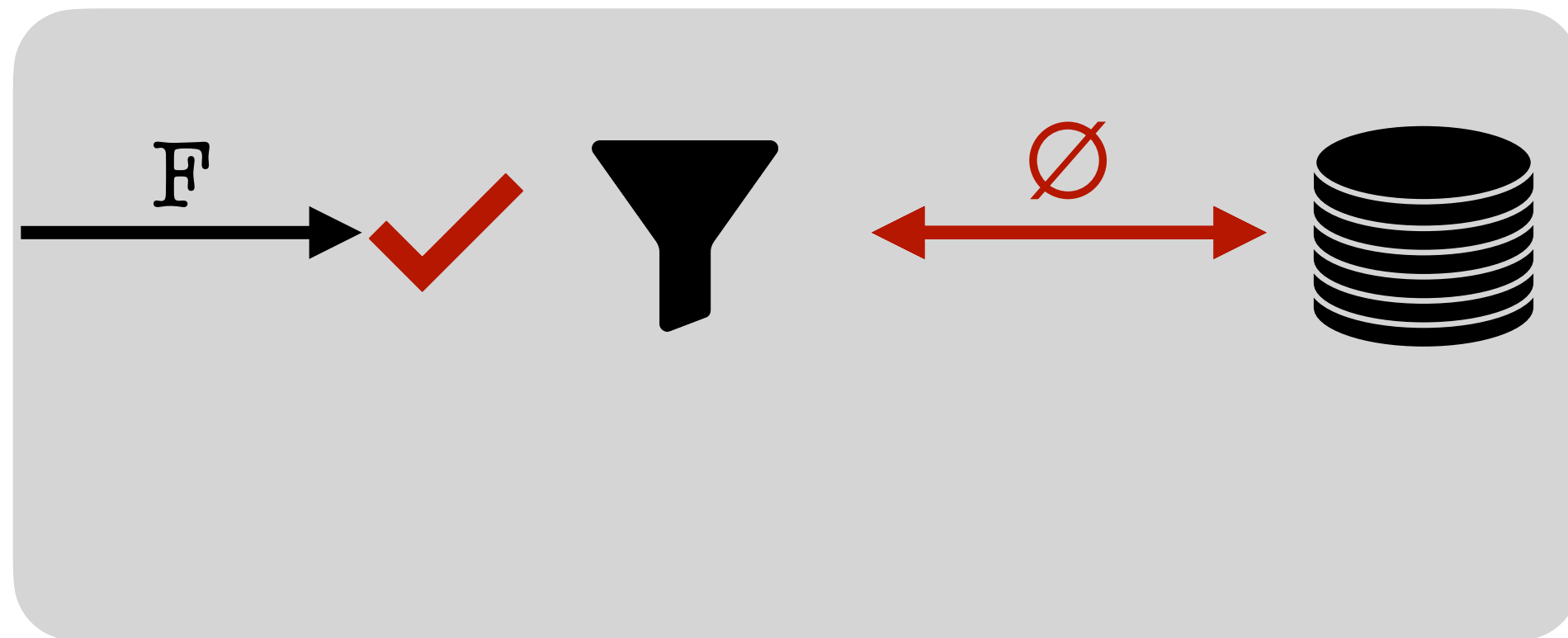


Adversarial workloads

Attacker replays false positives repeatedly

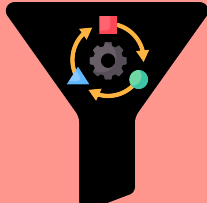
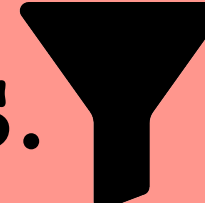
Adapting has overheads!

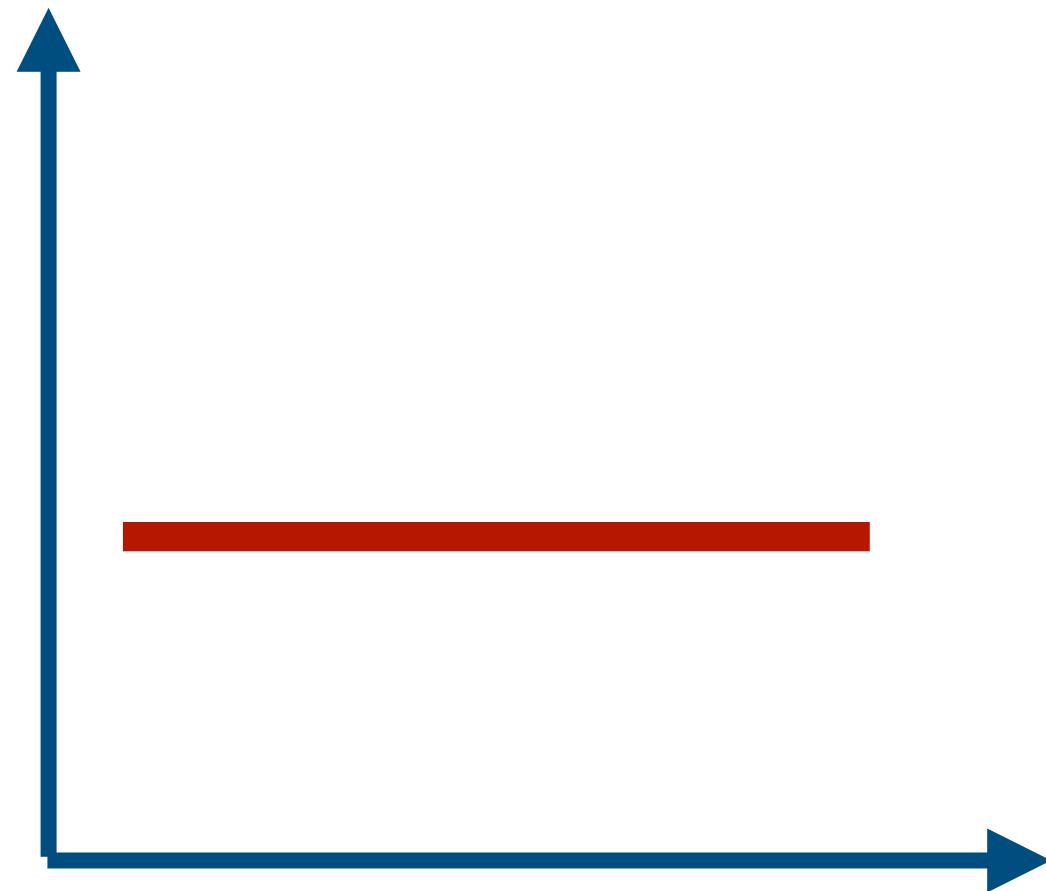
Adaptations **require additional I/Os** beyond the false positive check



This overhead is beneficial only if that false positive query repeats later!

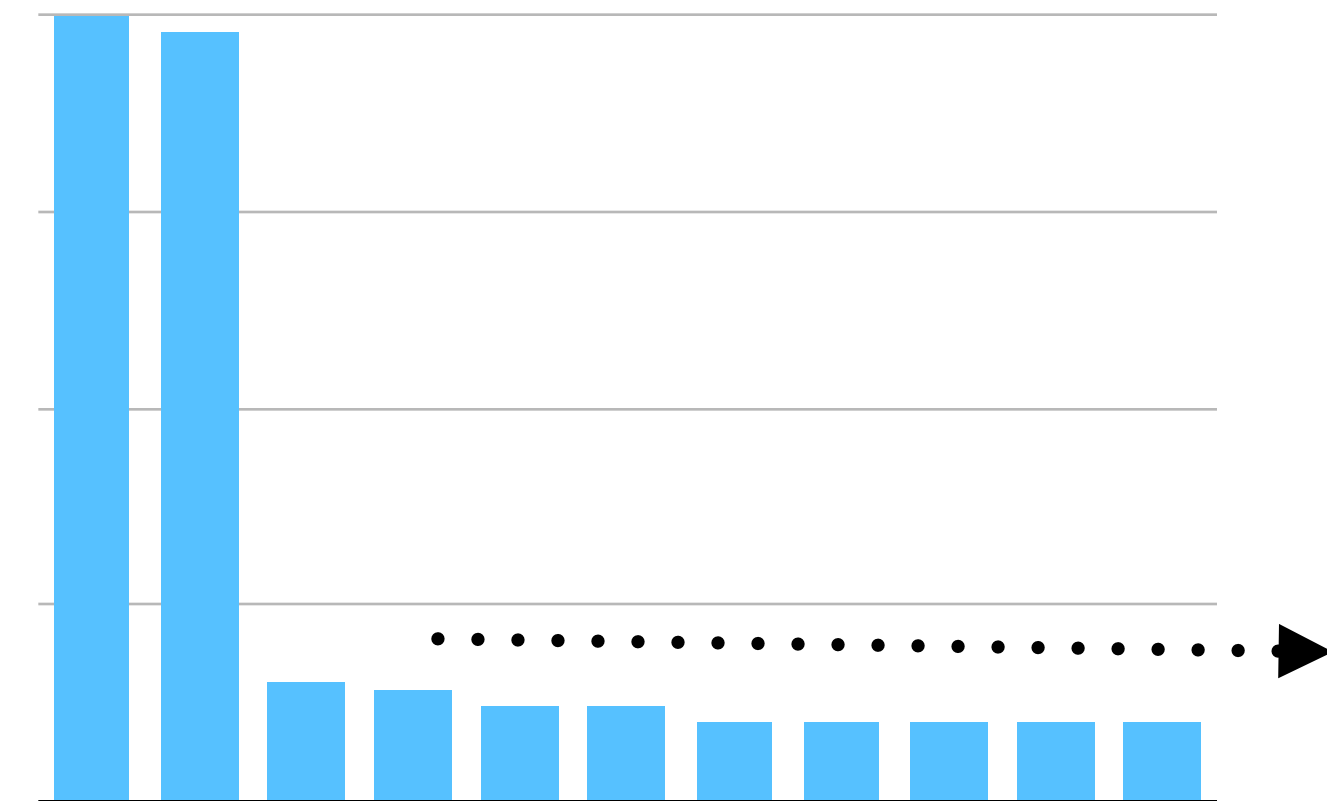
Adaptivity is an overhead when false positives don't repeat

 Vs.  : **0.5 - 0.9x** (Throughput)



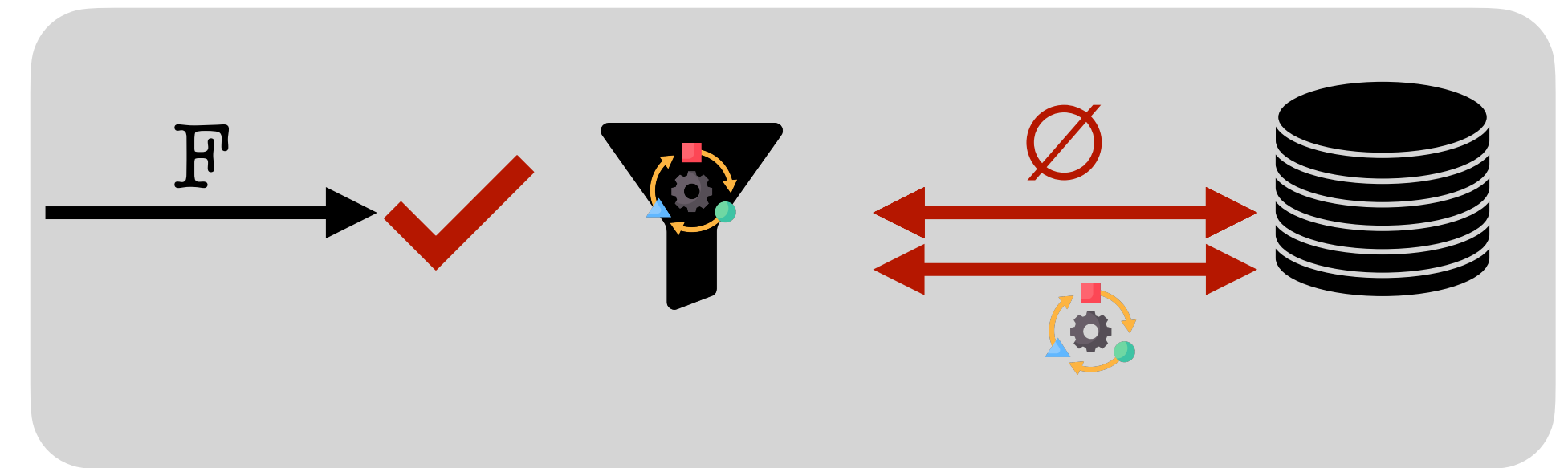
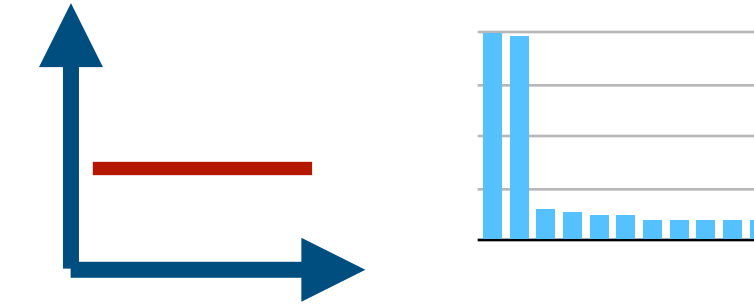
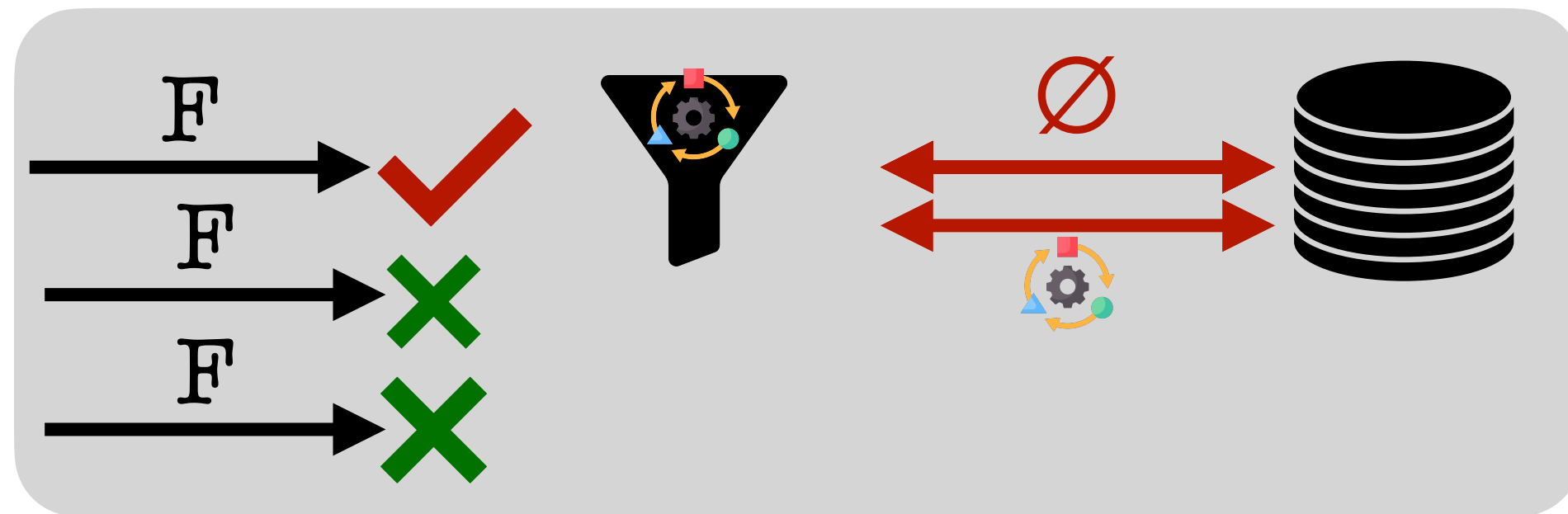
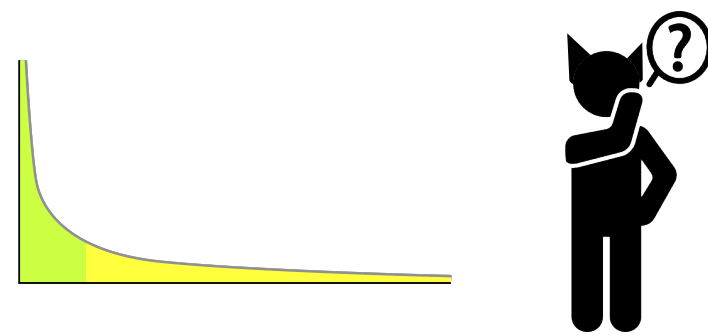
Uniform random workloads

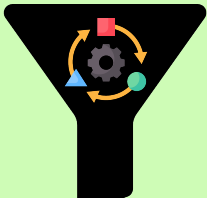
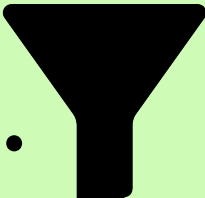
Very unlikely to repeat a false positive query

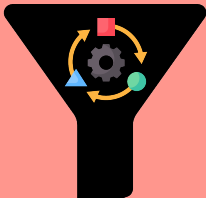
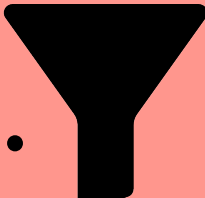


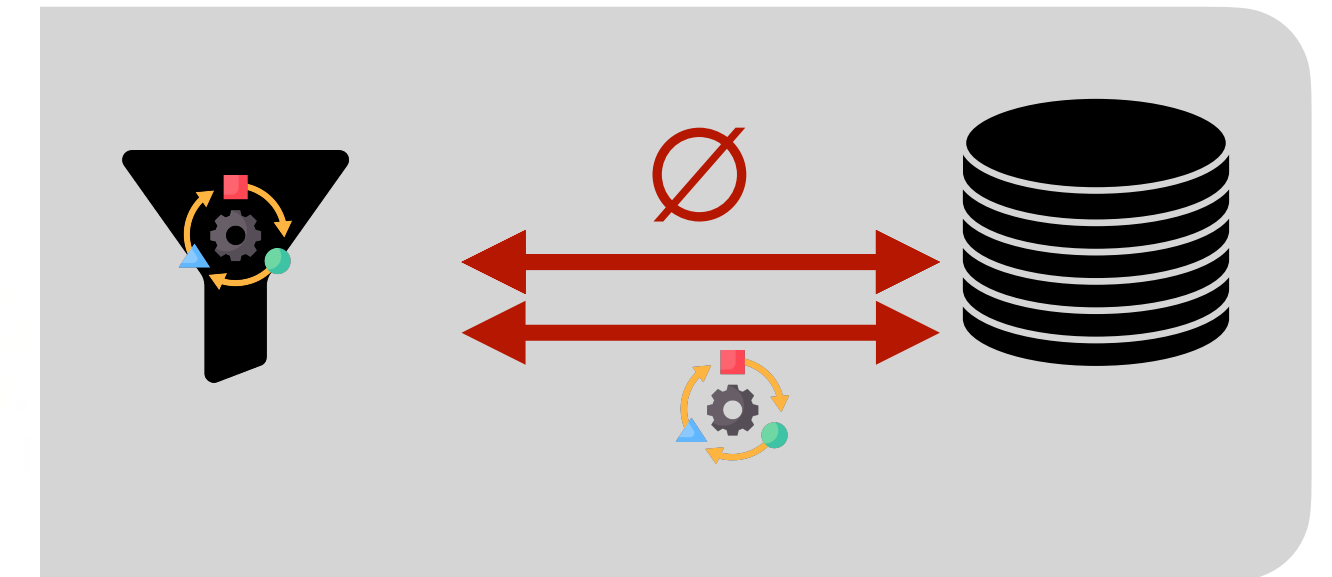
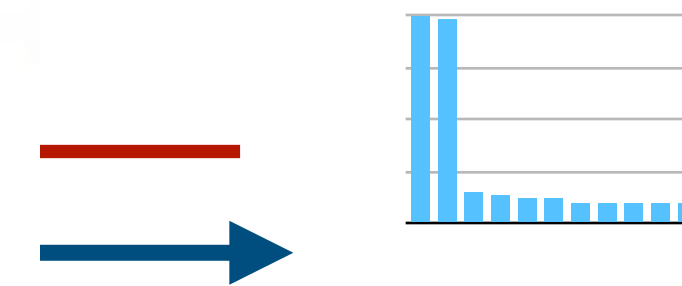
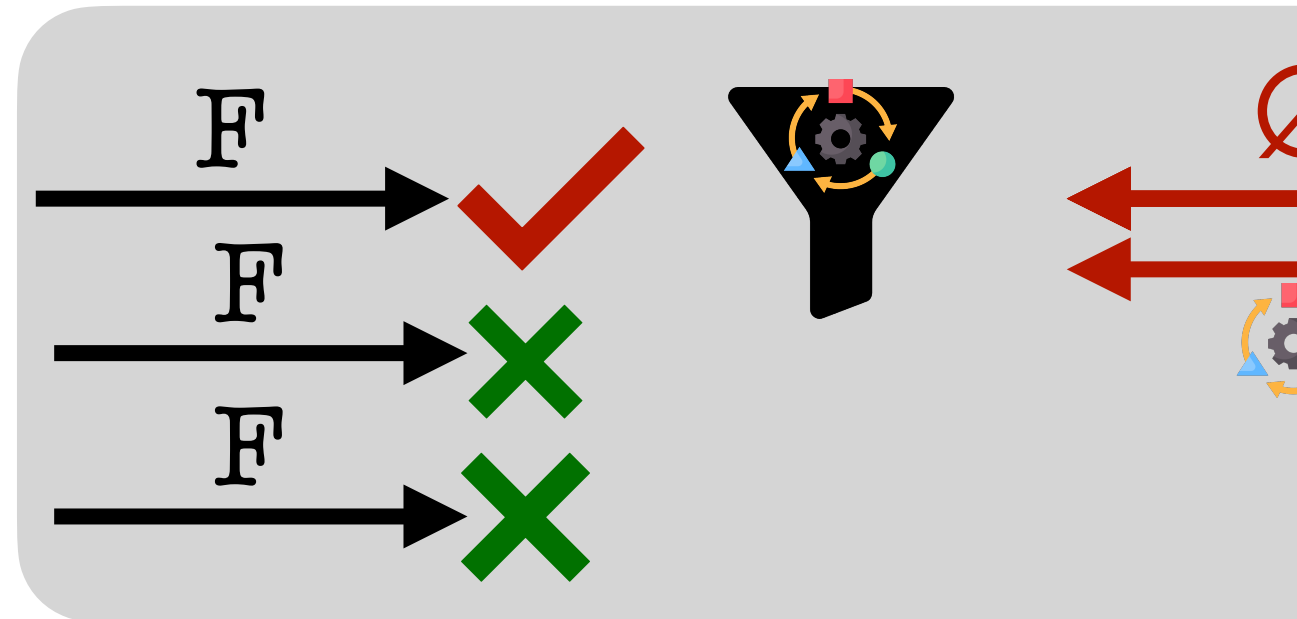
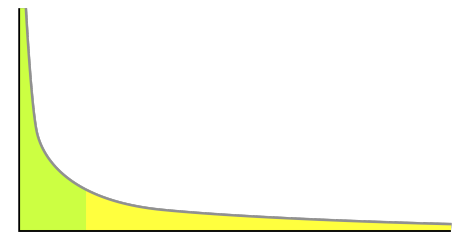
Heavy-tailed workloads

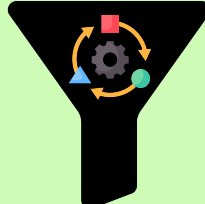
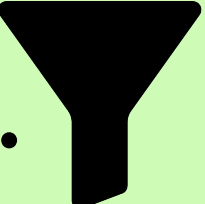
Many one-off false positives



 Vs.  : **1.3 - 12x** (Throughput)

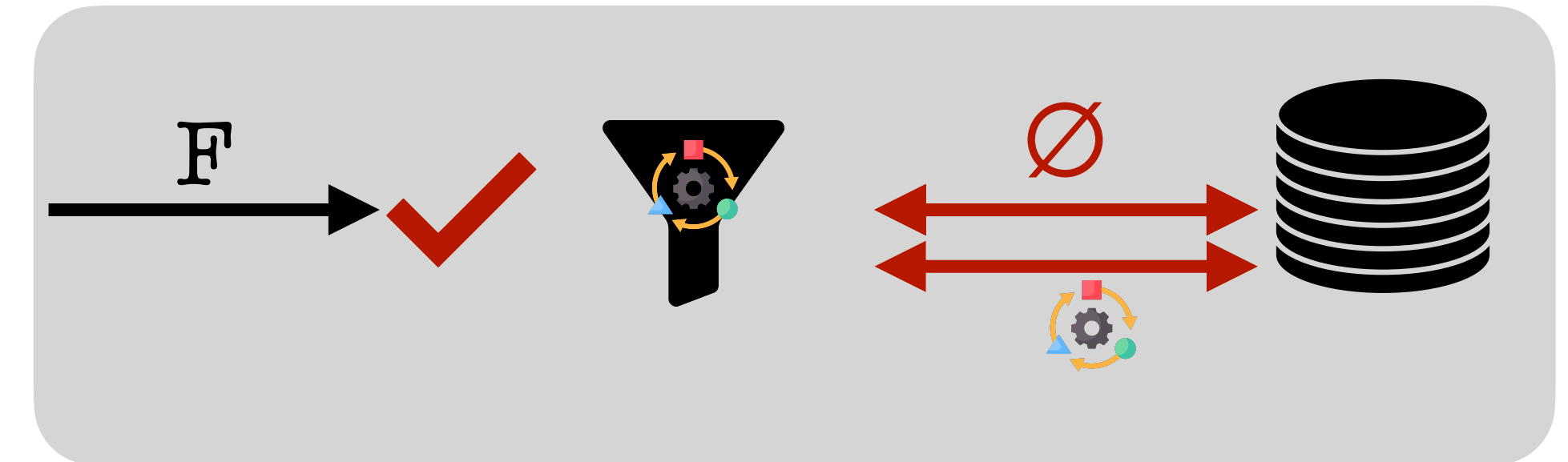
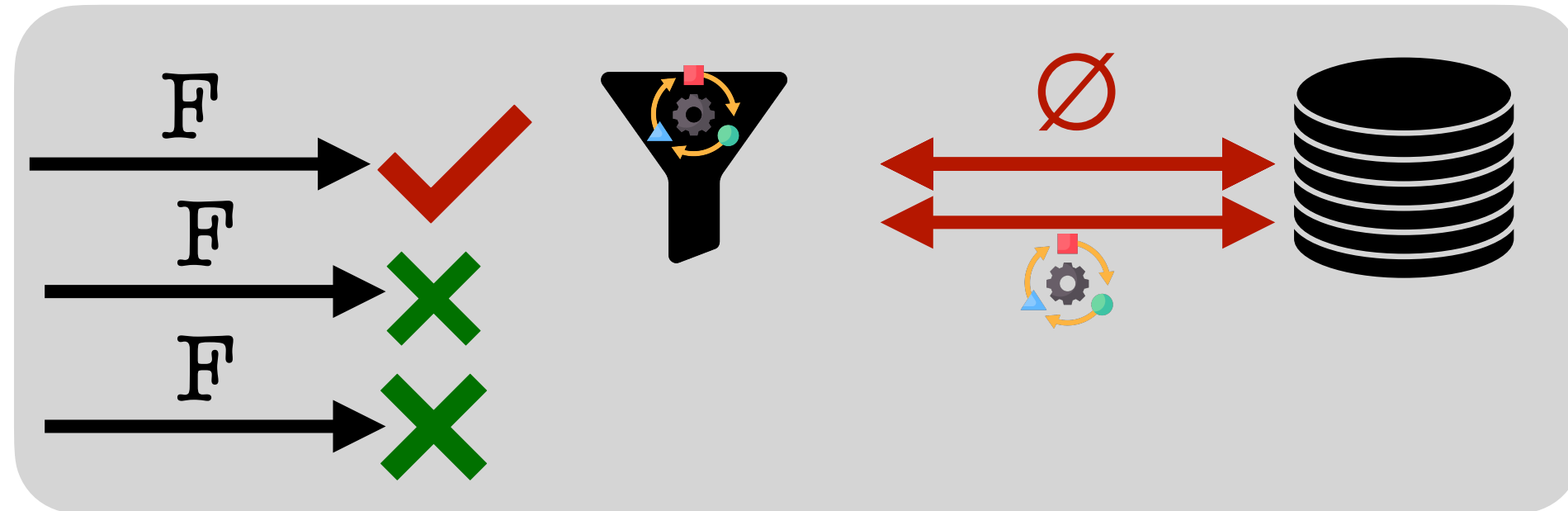
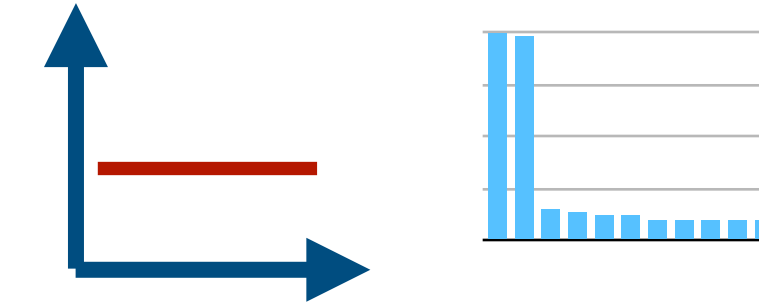
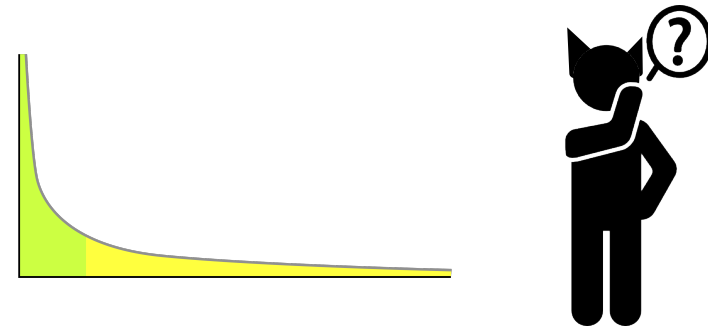
 Vs.  : **0.5 - 0.9x** (Throughput)

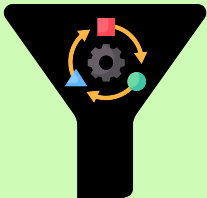
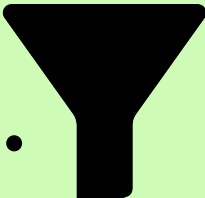


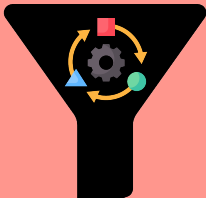
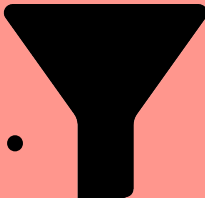
 Vs.  : **1.3 - 12x** (T



0.5 - 0.9x (Throughput)

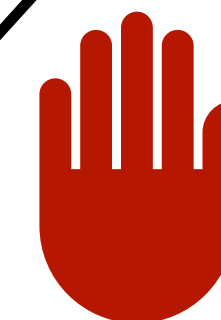
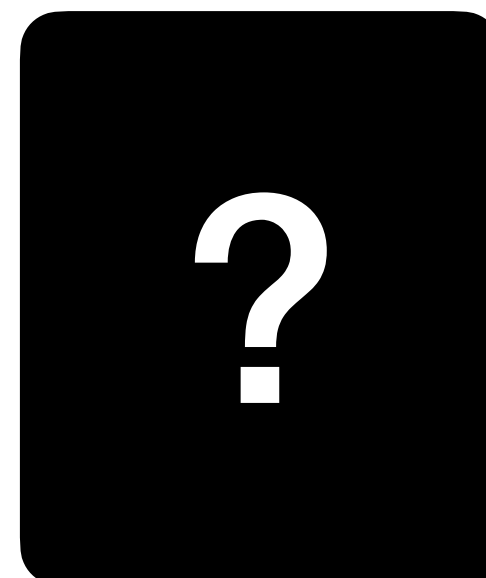


 Vs.  : **1.3 - 12x** (Throughput)

 Vs.  : **0.5 - 0.9x** (Throughput)

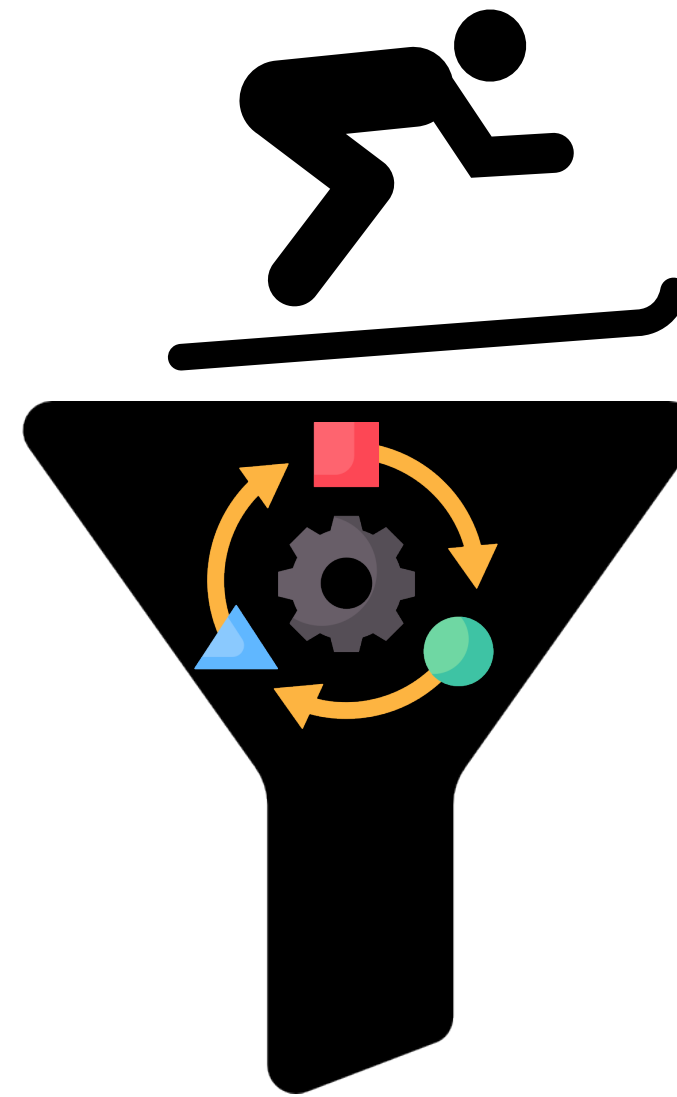


Benefits of adaptivity...



...without the risks!

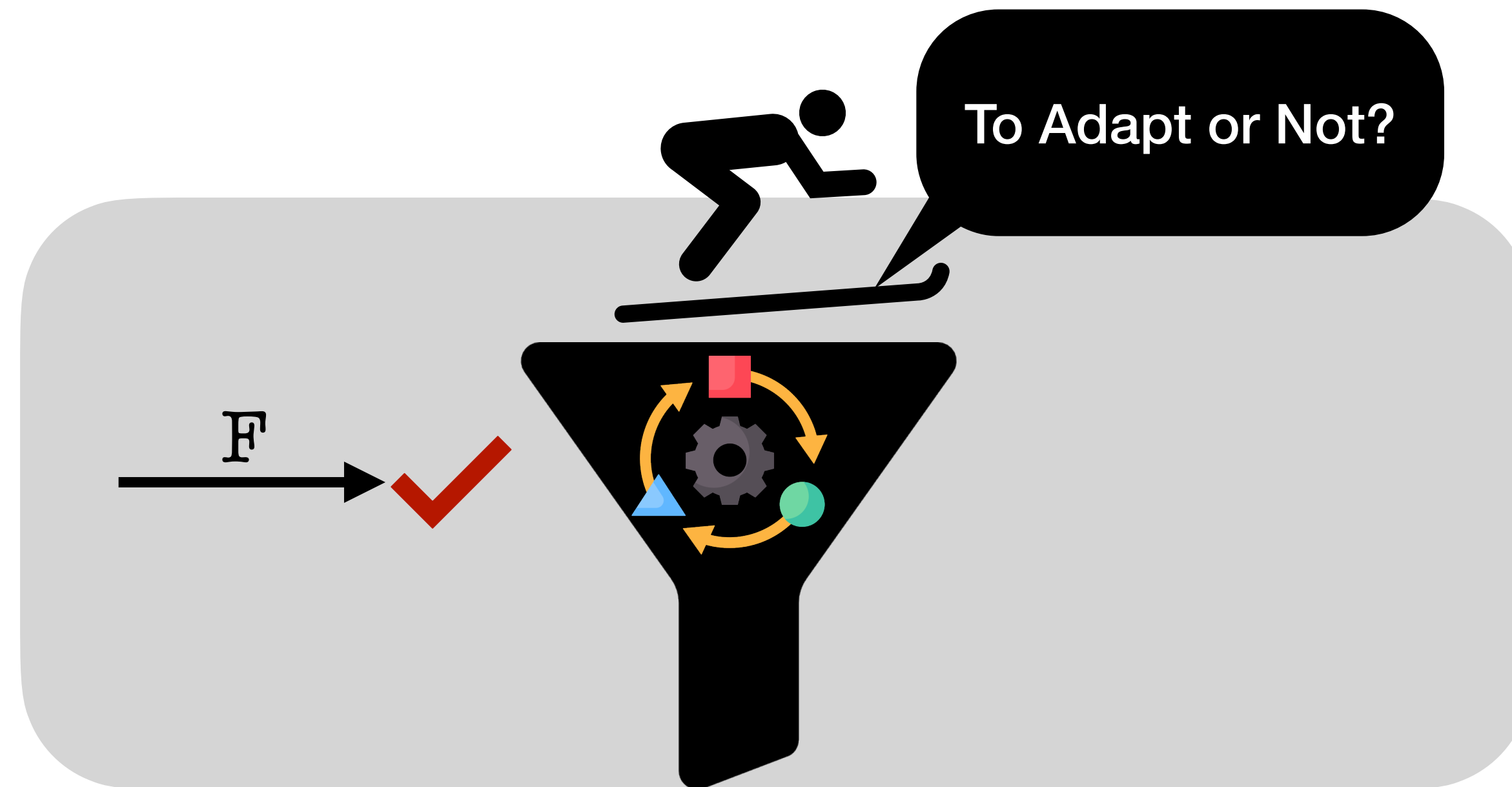
The Ski-Adaptive Quotient Filter (SkiQF)



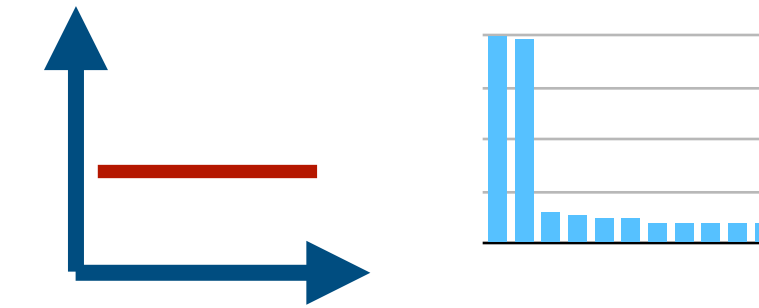
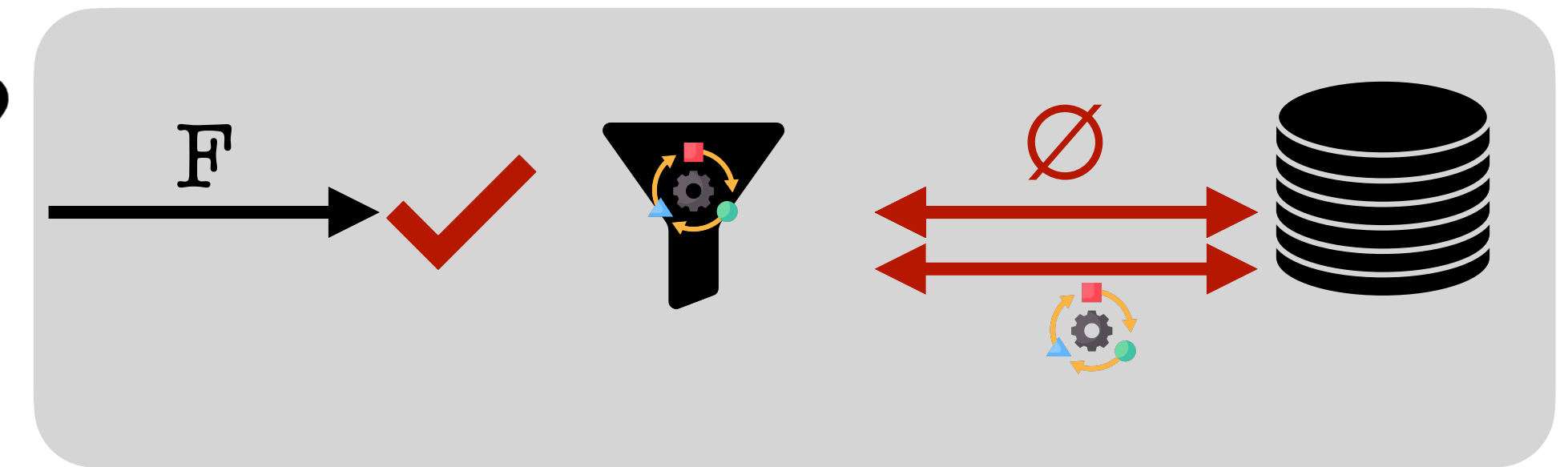
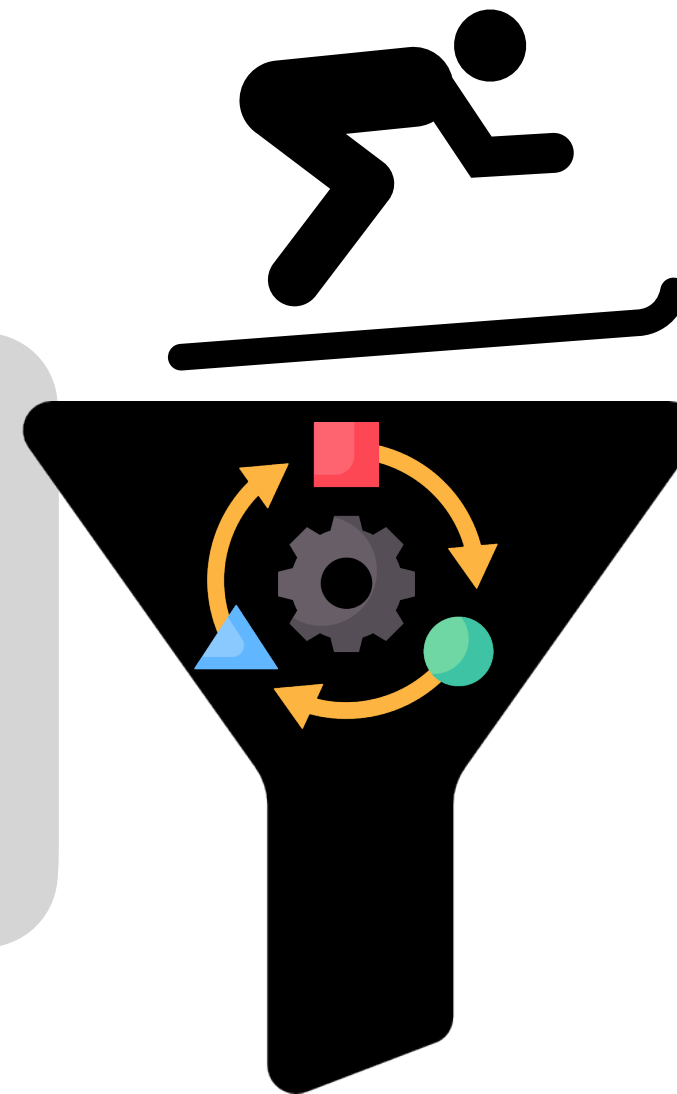
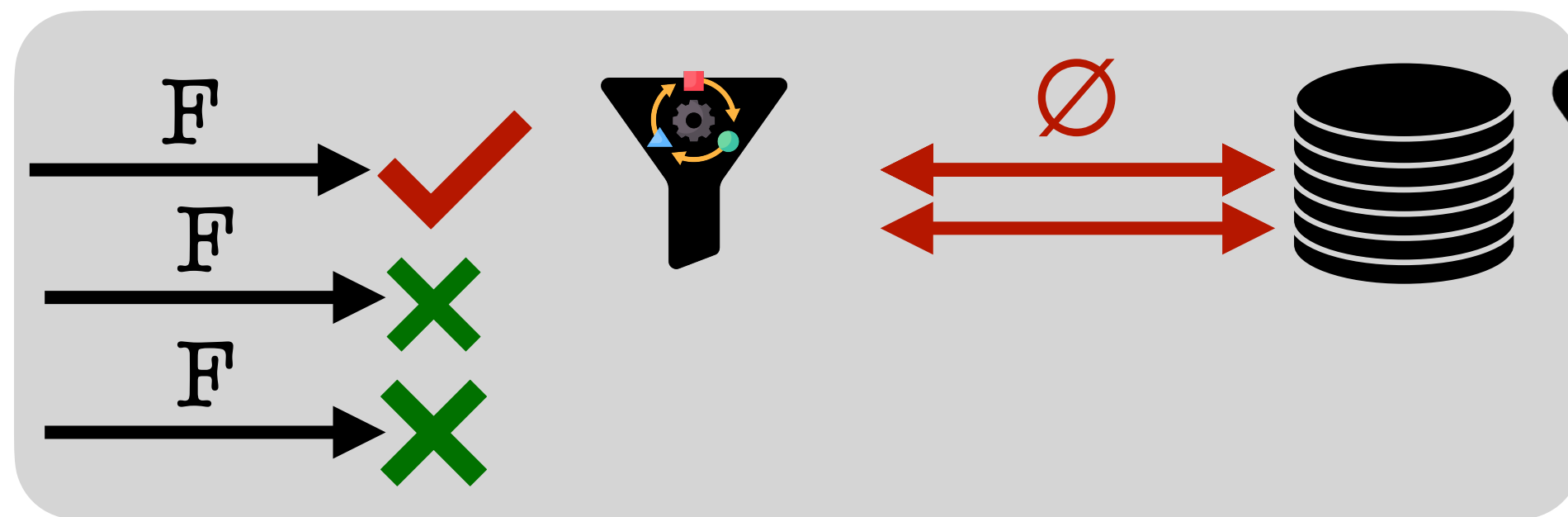
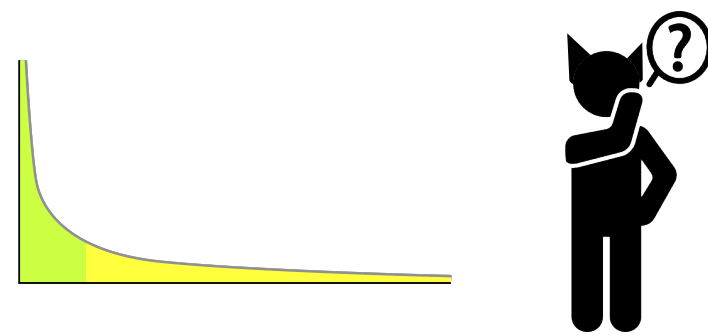
An **online-adaptive** filter that provides:

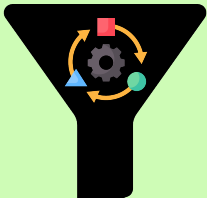
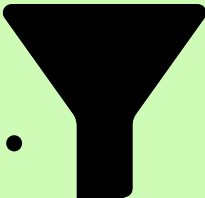
- ✓ **Stable performance** across workloads
- ✓ **No workload analysis** required
- ✓ **Minimal** memory/CPU overhead

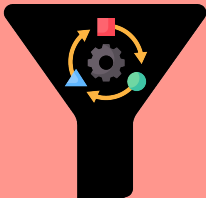
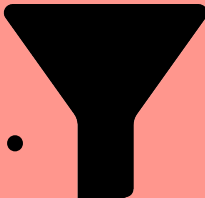
SkiQF poses adaptivity as a decision problem

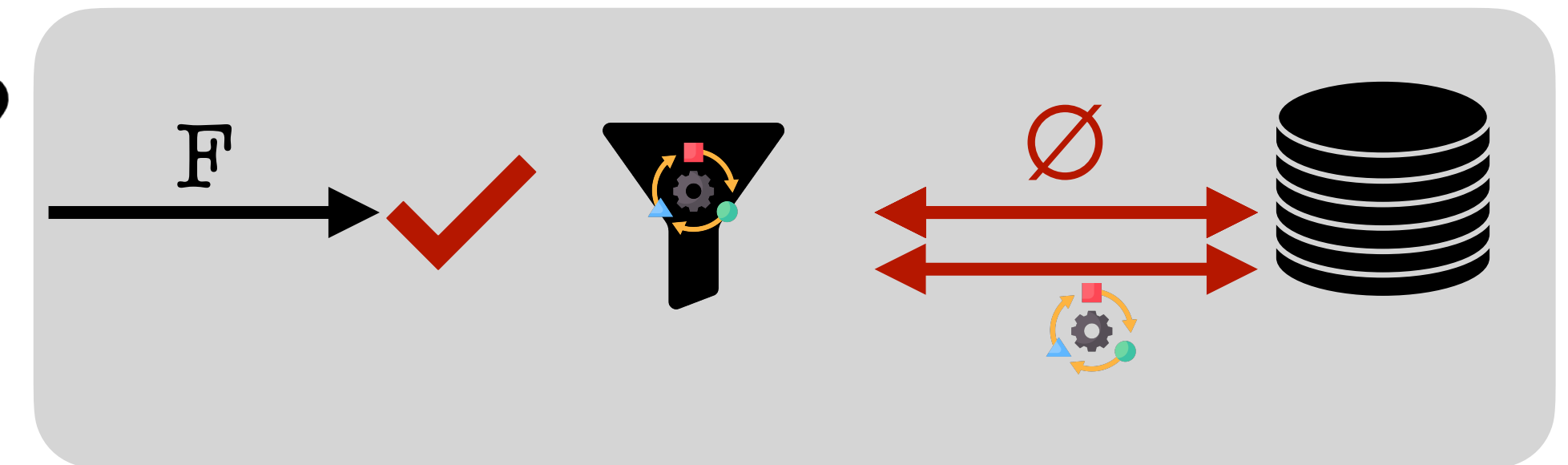
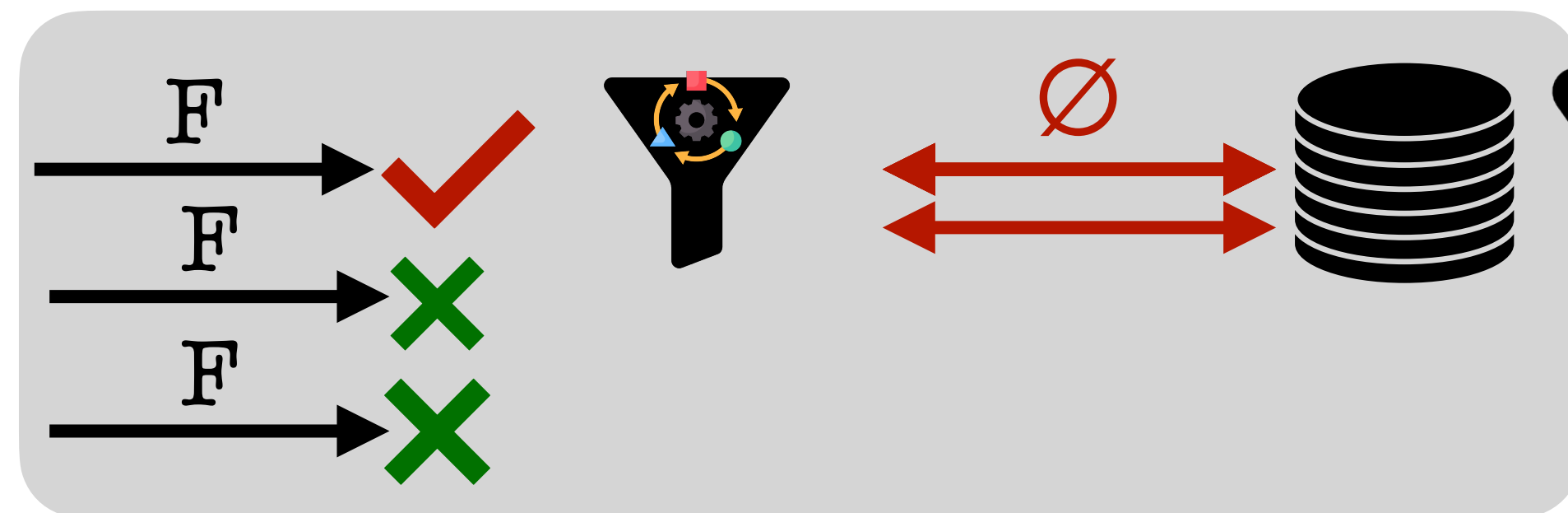
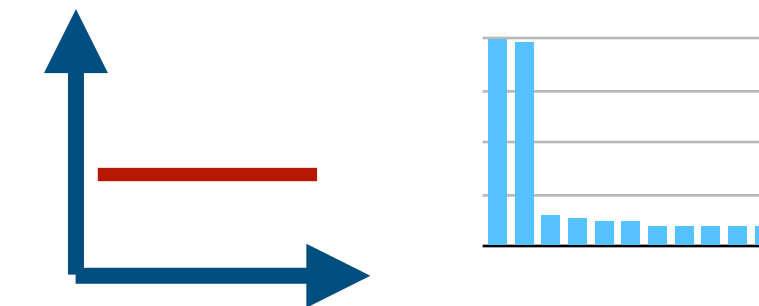
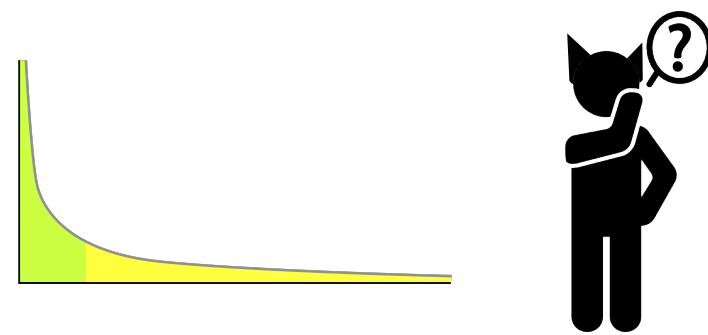


The **SkiQF** makes an **online decision** on **whether to adapt** a false positive

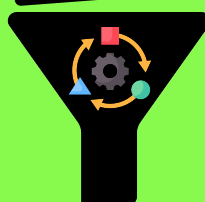


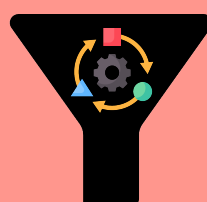
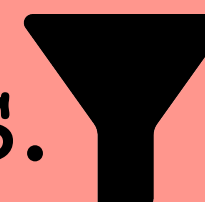
 Vs.  : **1.3 - 12x** (Throughput)

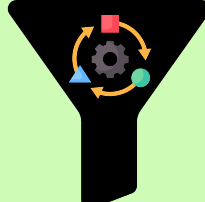
 Vs.  : **0.5 - 0.9x** (Throughput)

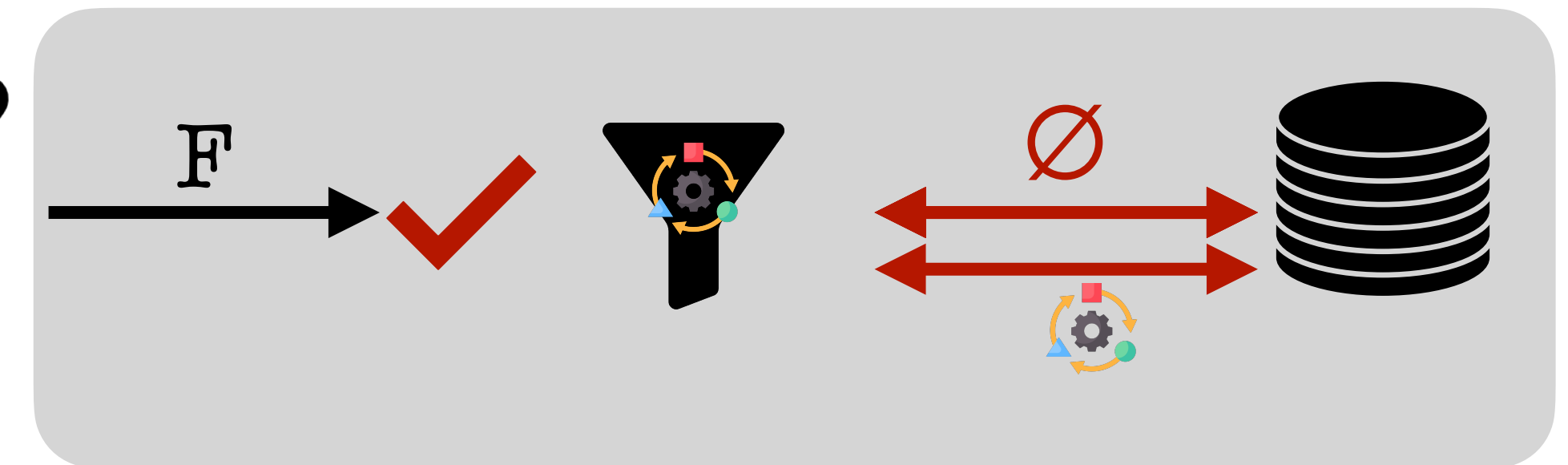
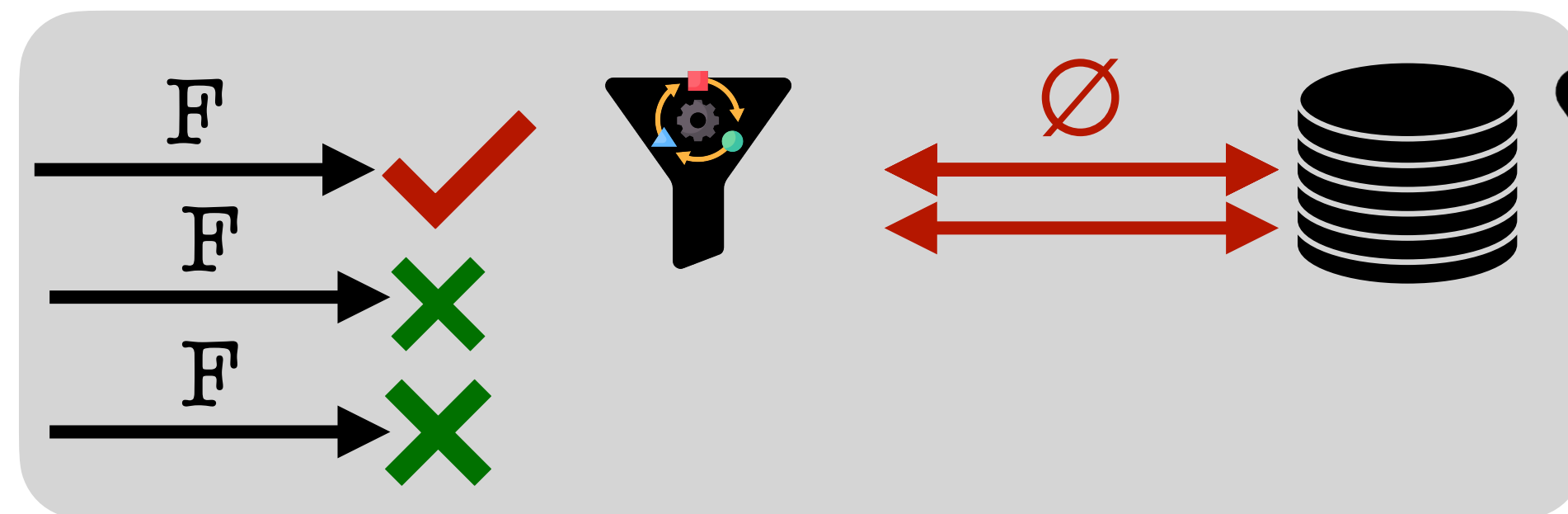
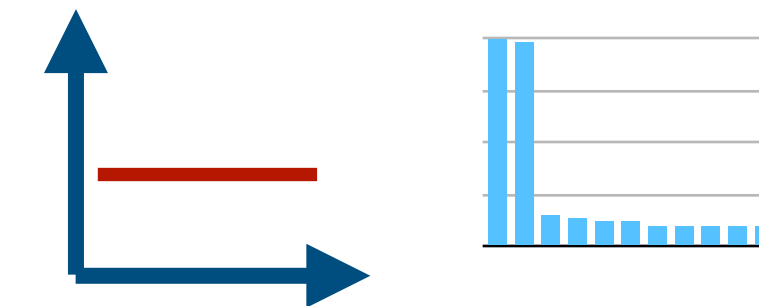
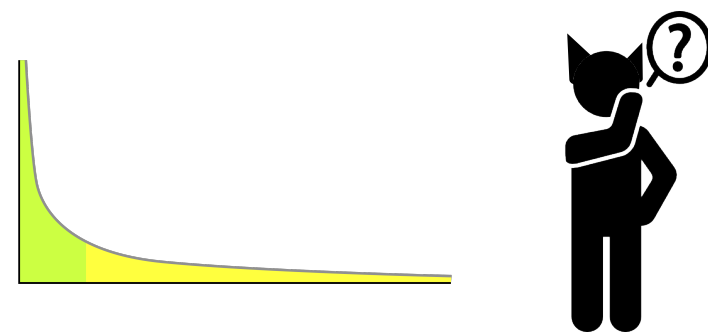


Slightly lower throughput...

 Vs.  : **1.2 - 10x** (Throughput)

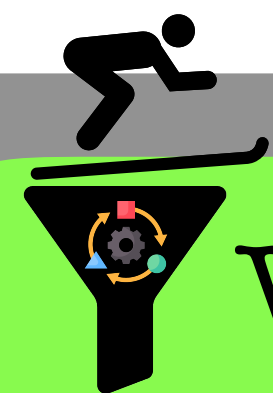
 Vs.  : **0.5 - 0.9x** (Throughput)

 Vs.  : **1.3 - 12x** (Throughput)

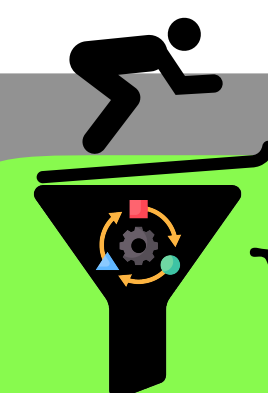


Slightly lower throughput...

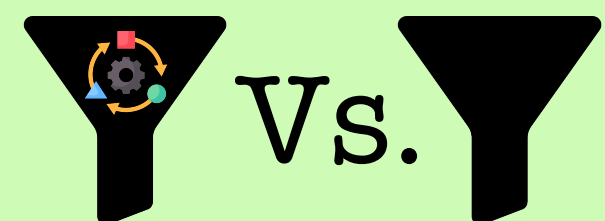
...but more robust!

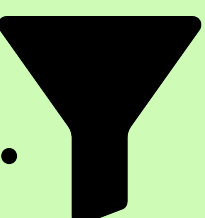


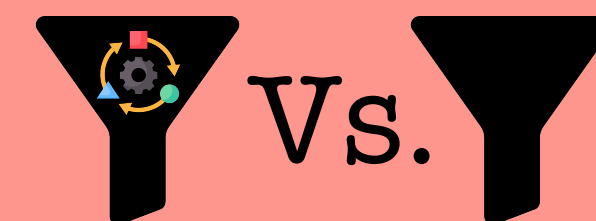
Vs.  : 1.2 - 10x (Throughput)

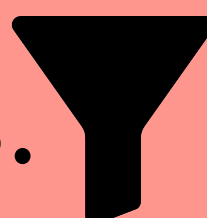


Vs.  : 1.0x (Throughput)

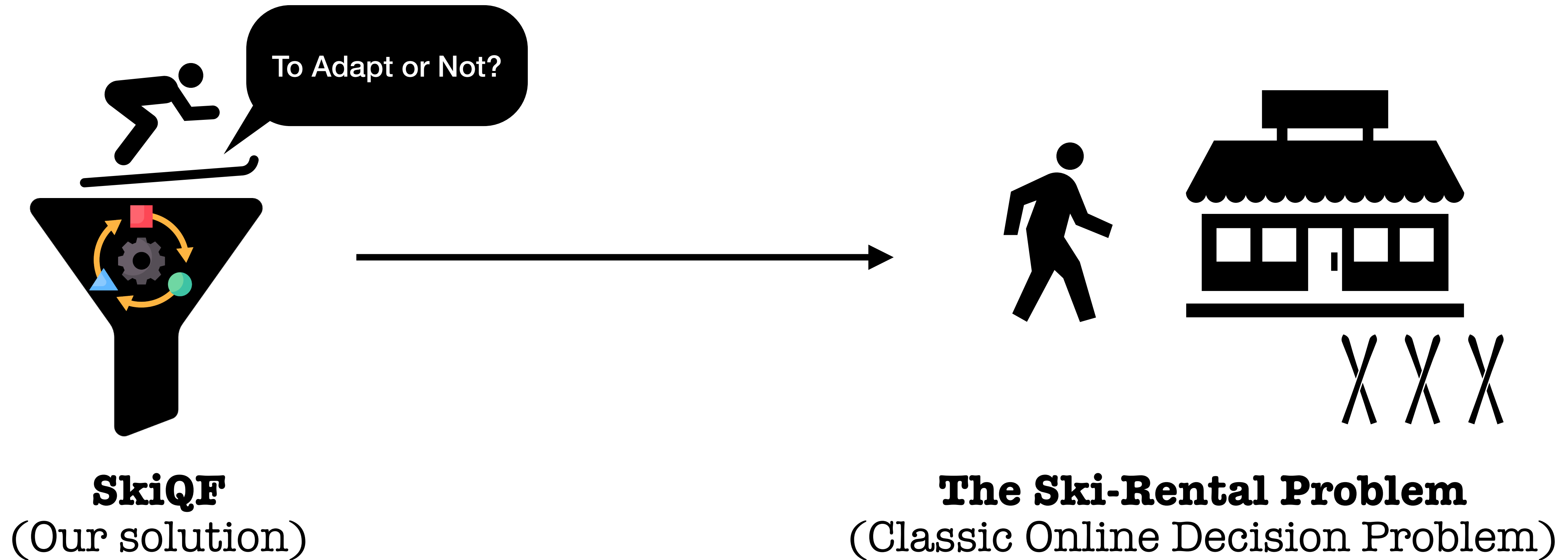


Vs.  : 1.3 - 12x (Throughput)

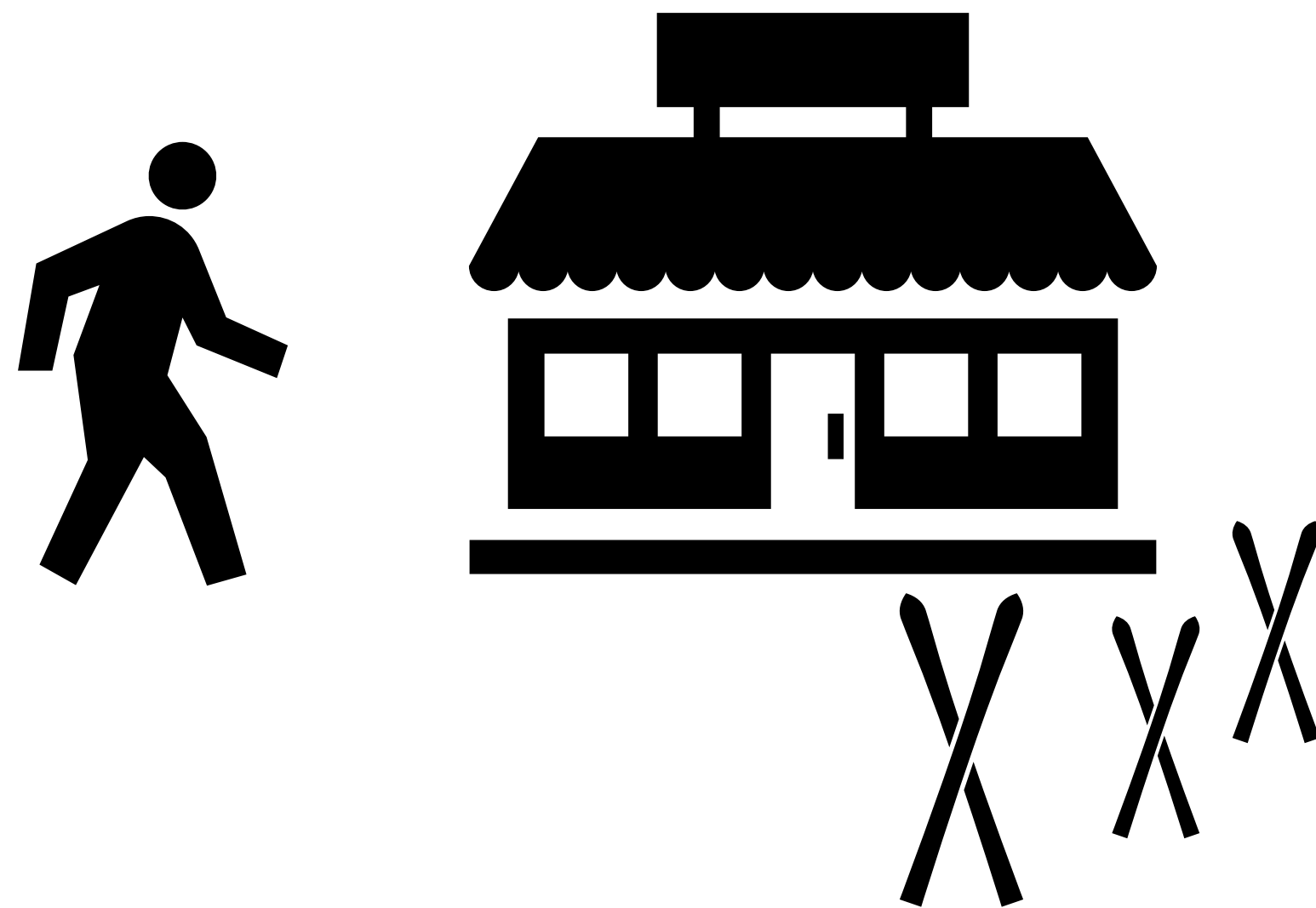


Vs.  : 0.5 - 0.9x (Throughput)

The SkiQF and the ski-rental problem



The ski-rental problem



Rent

50\$ per day

Buy

500\$ (one-time cost)

How many ski-days???

The ski-rental decision algorithm

Algorithm

Buy on $\lfloor B/r \rfloor$ day,
until then rent.

$\lfloor B/r \rfloor$: Break-even day

B : Buying cost

r : renting cost

Only depends on B, r !

The ski-rental decision algorithm

Algorithm

Buy on $\lfloor B/r \rfloor$ day,
until then rent.

$\lfloor B/r \rfloor$: Break-even day

B : Buying cost

r : renting cost

Only depends on B, r !

Rent

50\$ per day

Buy

500\$ (one-time cost)

$$\lfloor B/r \rfloor = \lfloor 500/50 \rfloor = 10$$

Rent until day 9,
Buy on day 10



Worst-case scenario using ski-rental algorithm

Worst-case: Stop skiing immediately after buying!



Ski-rental algorithm

$450\$ + 500\$ = \mathbf{950\$}$

(9 days renting, buy on 10th)



Optimal cost:

500\$

(Either buy on 1st day,
or rent for 10 days)

Worst-case scenario using ski-rental algorithm

Worst-case: Stop skiing immediately after buying!



Ski-rental algorithm

$$450\$ + 500\$ = \mathbf{950\$}$$

(9 days renting, buy on 10th)



Optimal cost:

500\$

(Either buy on 1st day,
or rent for 10 days)

How bad can it get?



$$\frac{950}{500} = 1.90$$

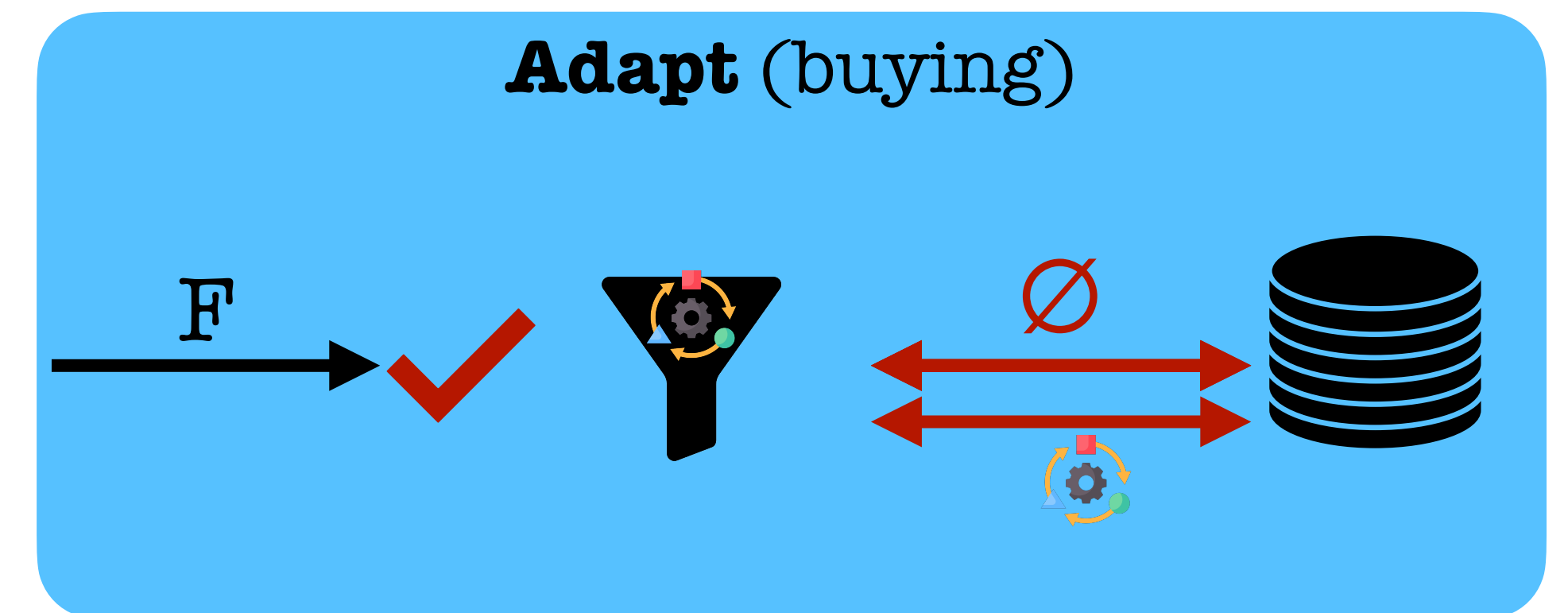
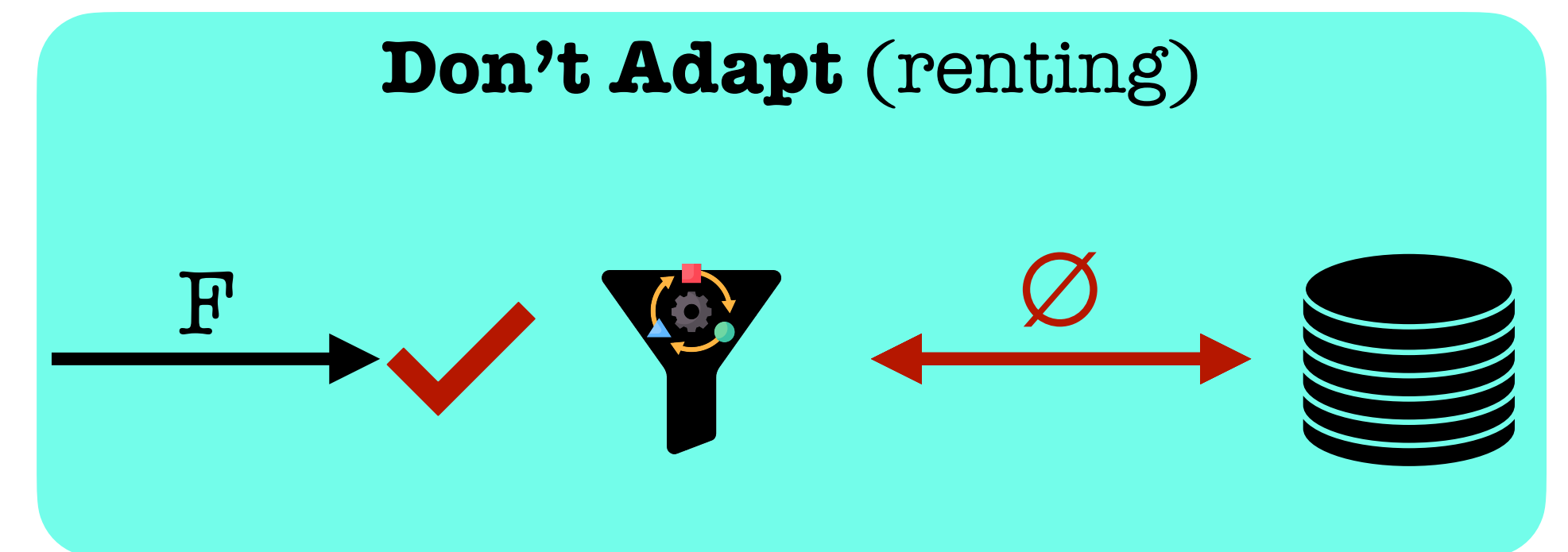
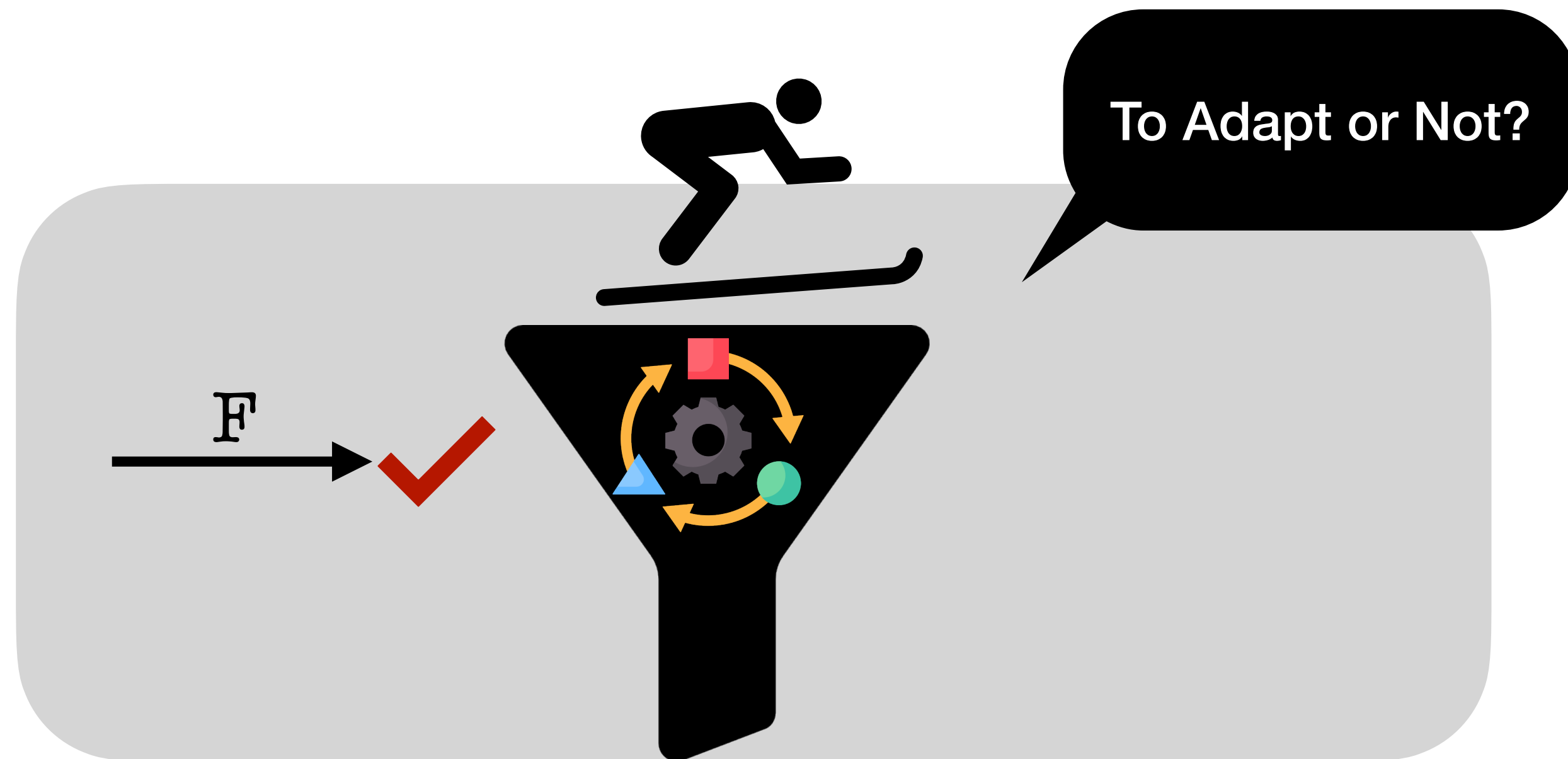
Ski-Rental algorithm guarantees 2-competitive cost

Competitive ratio : $\frac{\text{Online algorithm cost}}{\text{Optimal cost}}$

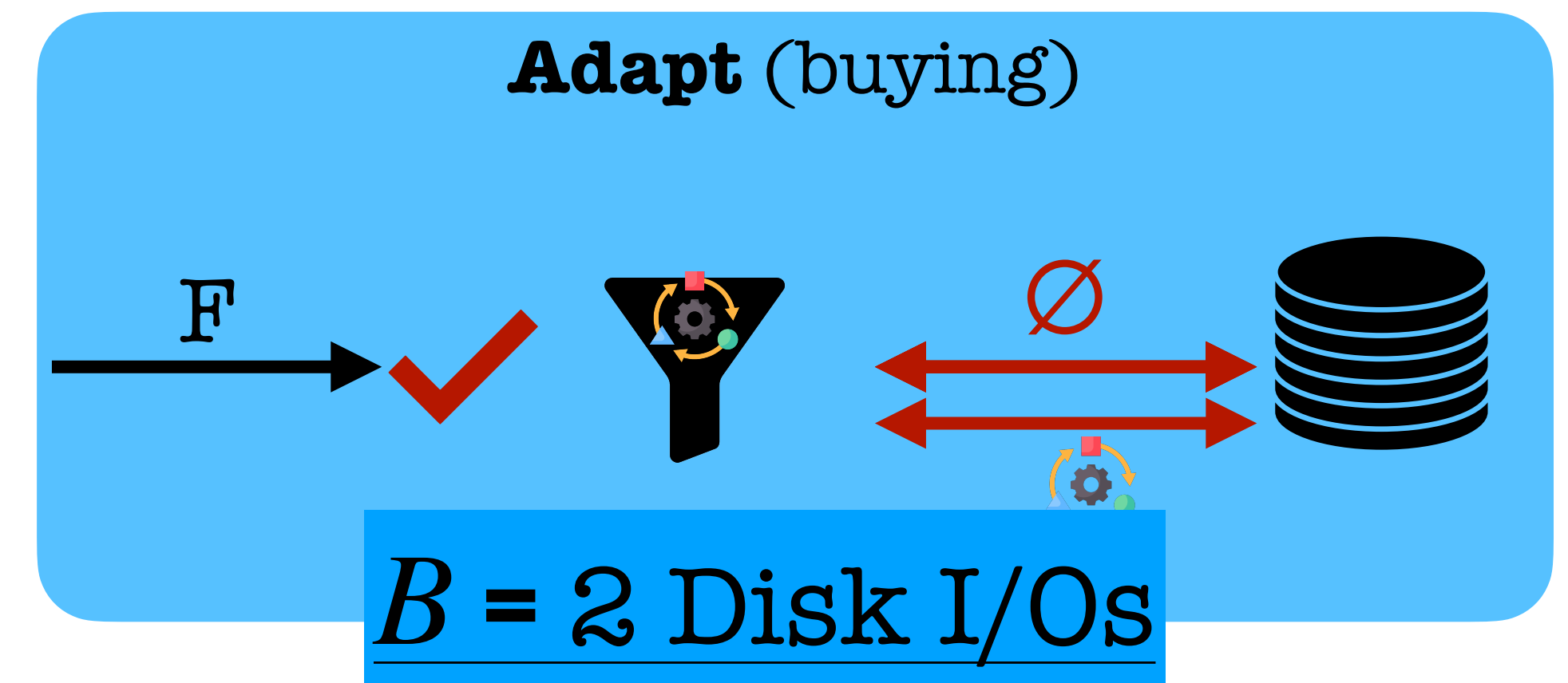
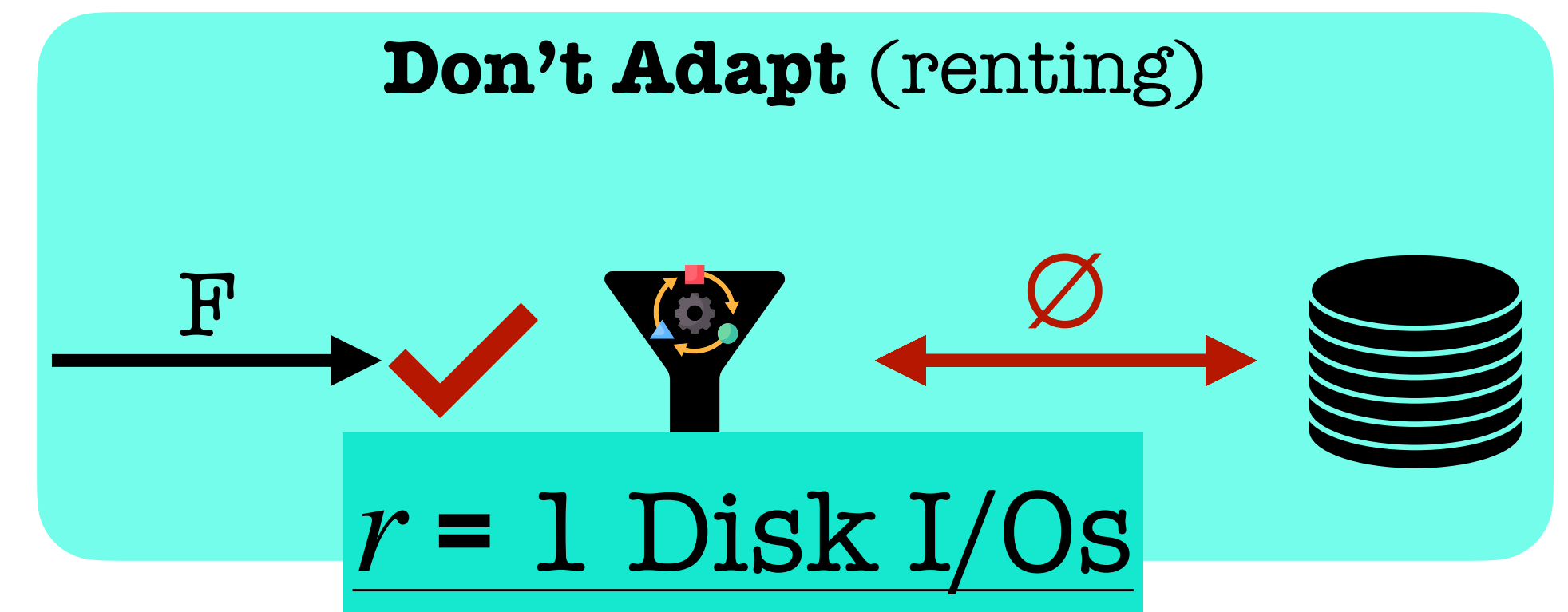
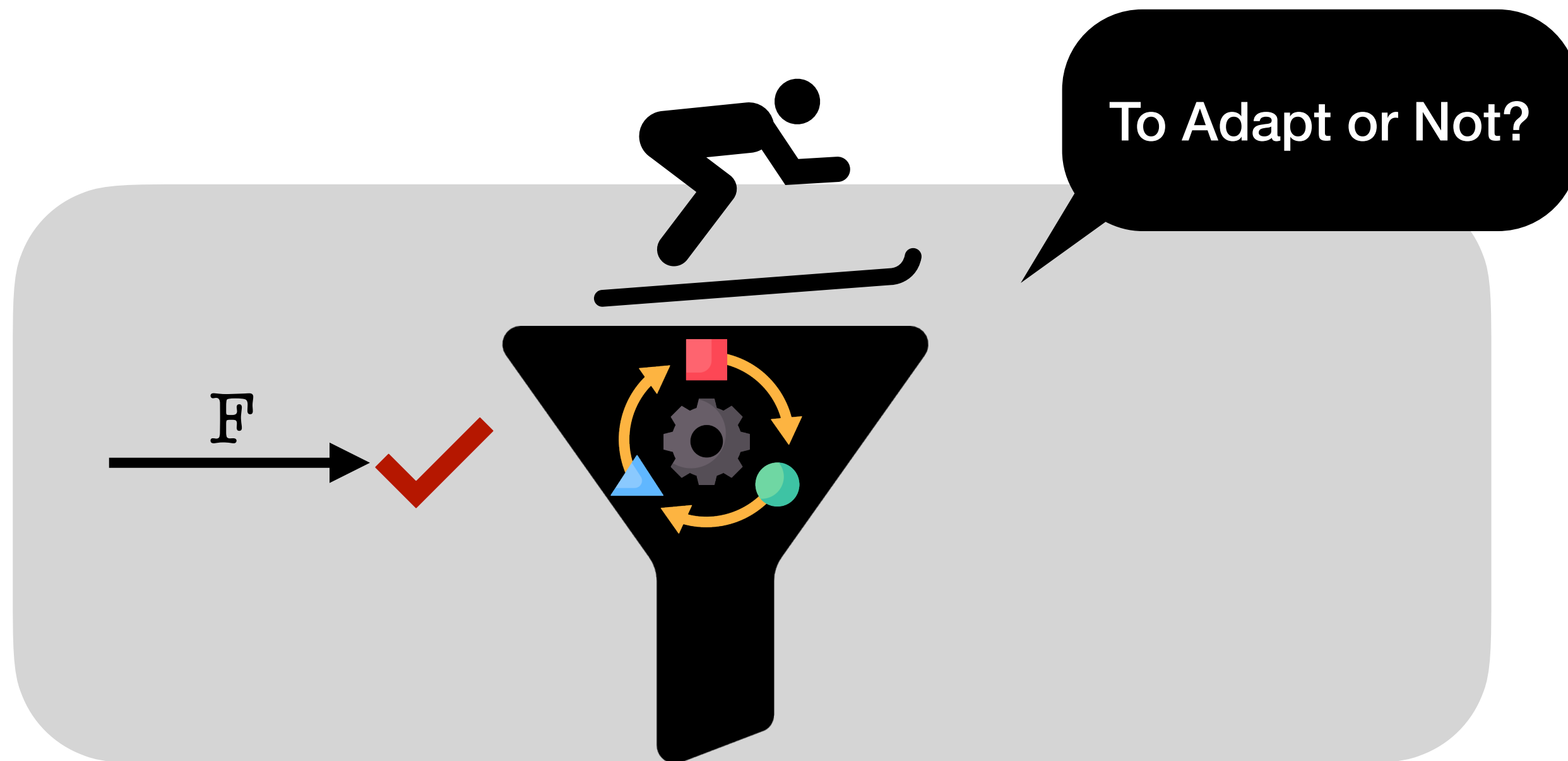


Ski-rental algorithm $\longrightarrow$ **Guarantees:**
Competitive ratio ≤ 2 ,
for any value of B, r
and inputs

Adaptivity as a ski-rental problem

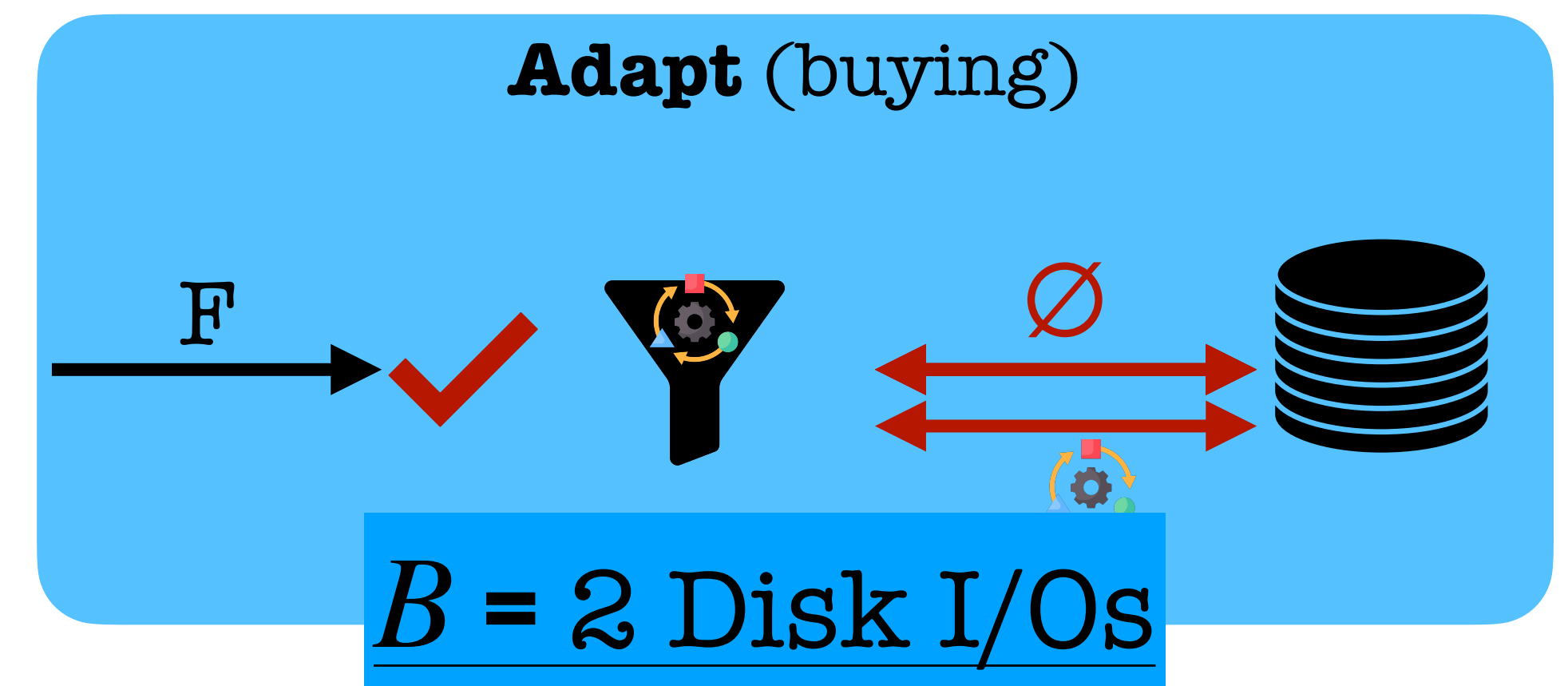
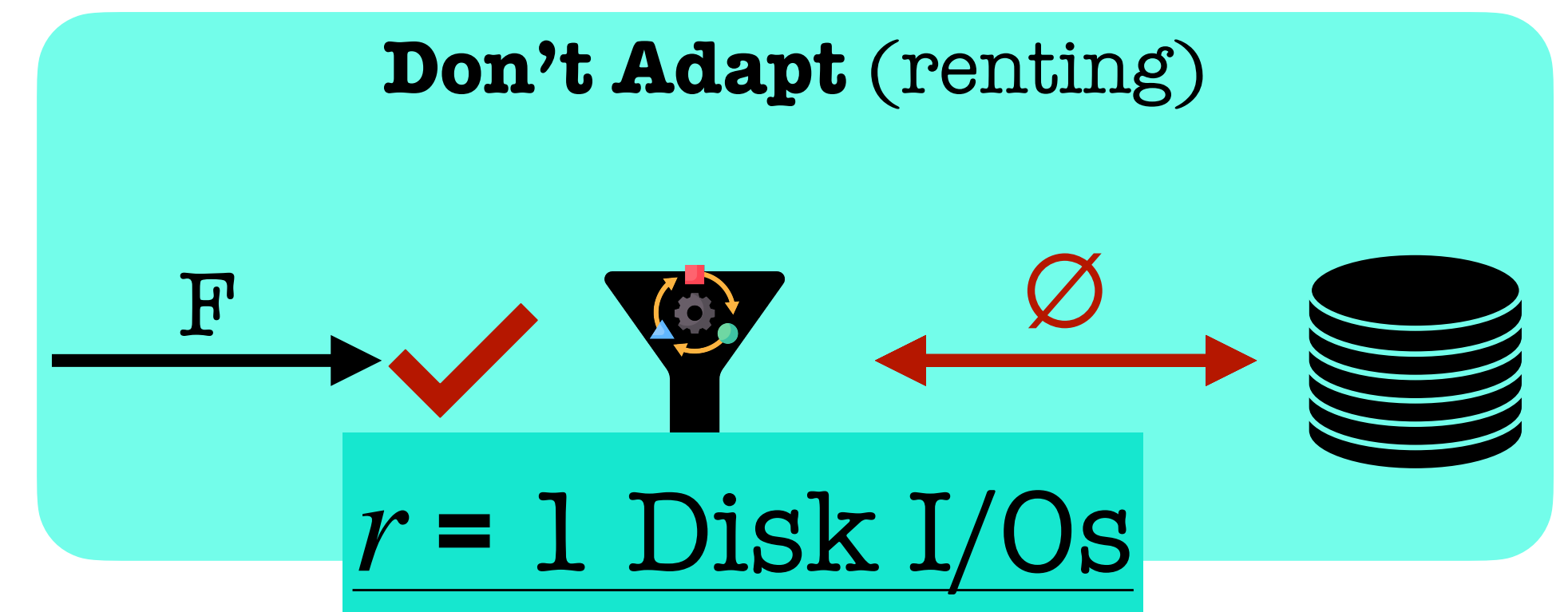
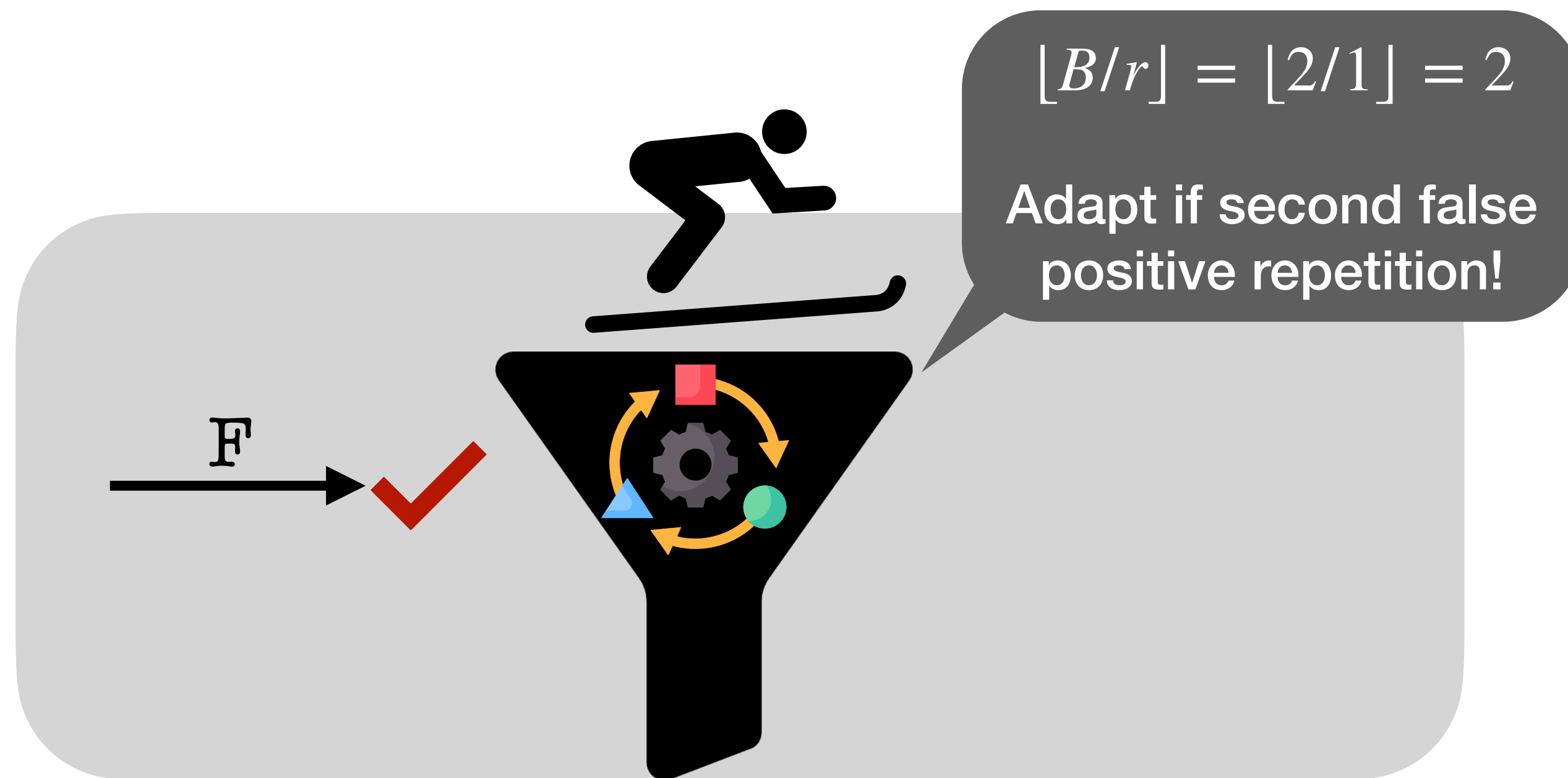


Adaptivity as a ski-rental problem



Simplification representing most common setup!

Applying the ski-rental algorithm to the SkiQF



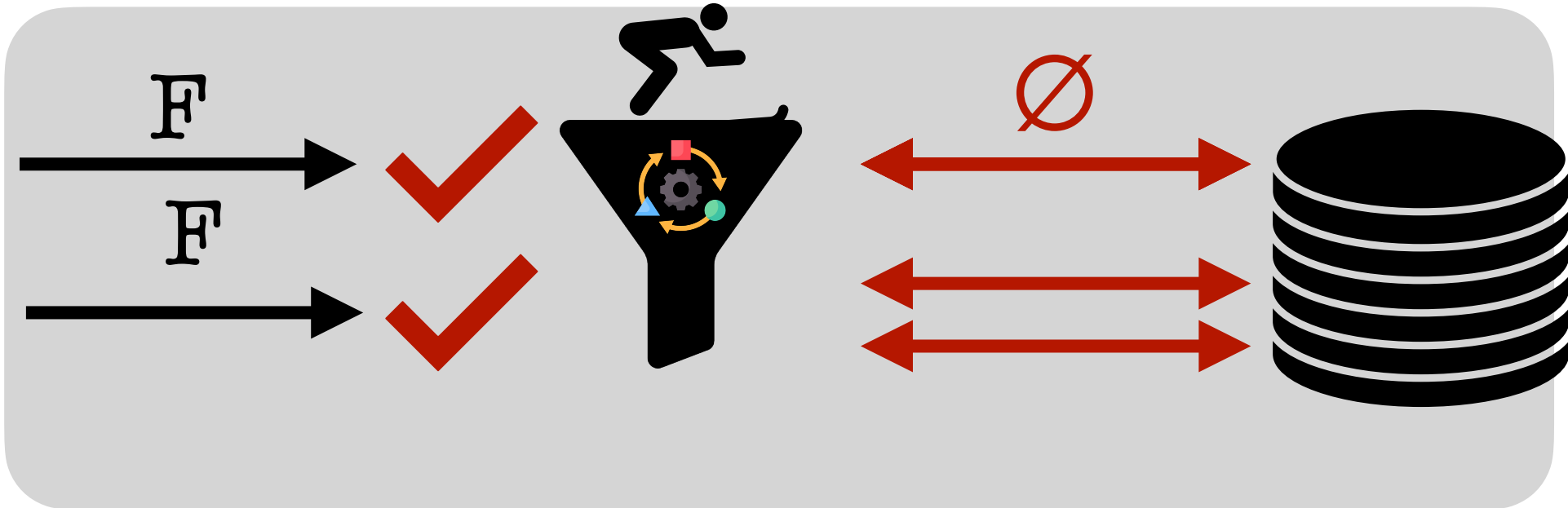
Simplification representing most common setup!

Competitive ratio of the SkiQF

Worst-case:

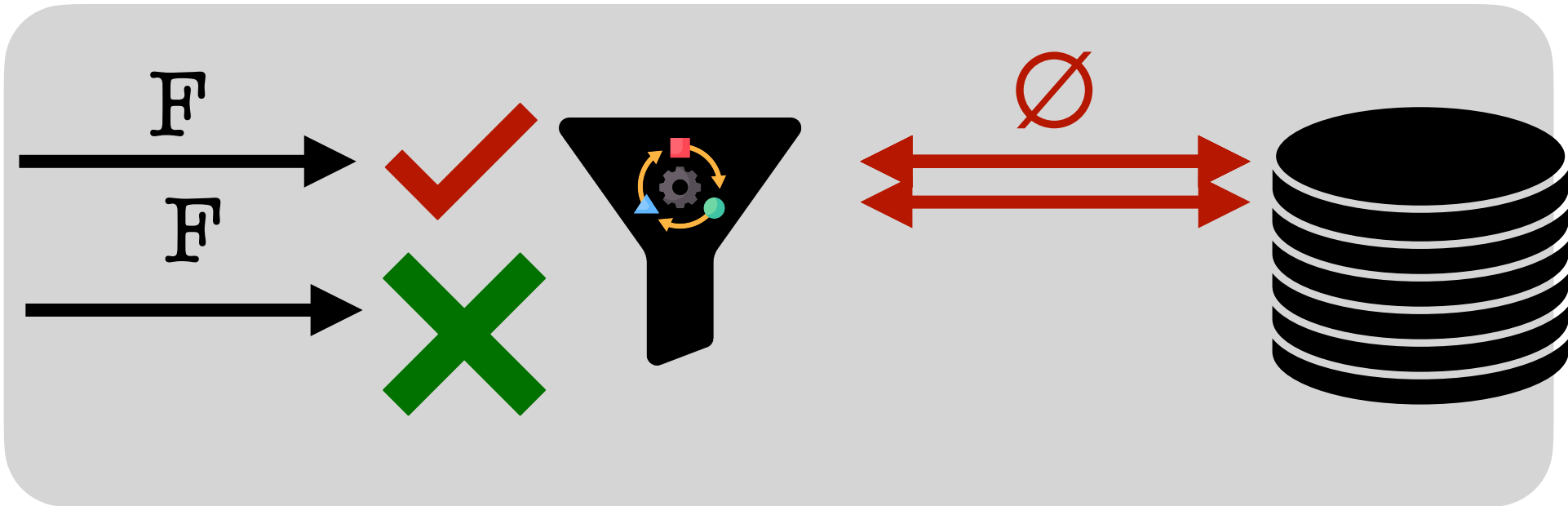
False positive does not repeat after adapting
(False positive repeats exactly twice)

SkiQF cost:

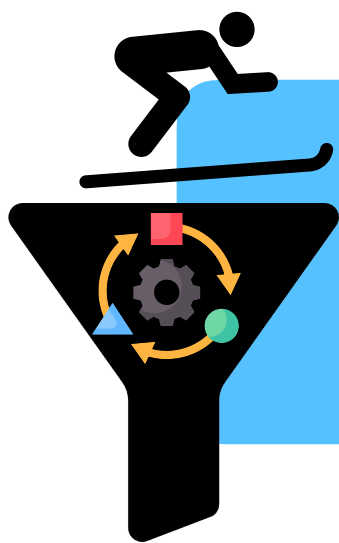


Cost = 3 Disk I/Os

Optimal cost:



Cost = 2 Disk I/Os

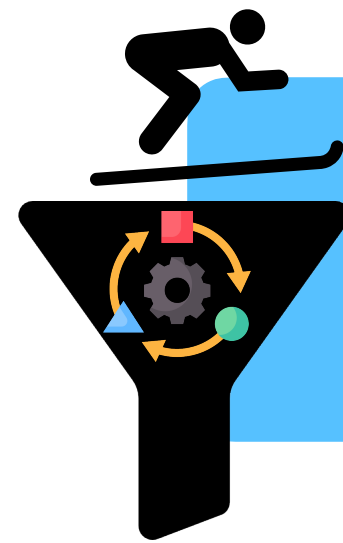


Competitive ratio: $\frac{3}{2} = 1.5$

Competitive ratio of adaptive and non-adaptive filters

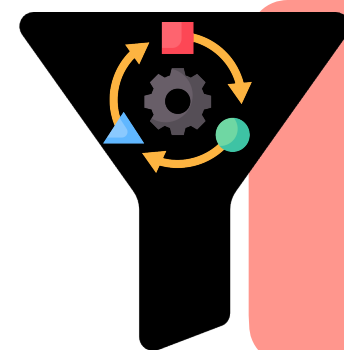
(When, $B=2$, $r=1$, Lower competitive-ratio is better)

SkiQF



Competitive ratio: 1.5

Adaptive Filter
(adapt immediately)



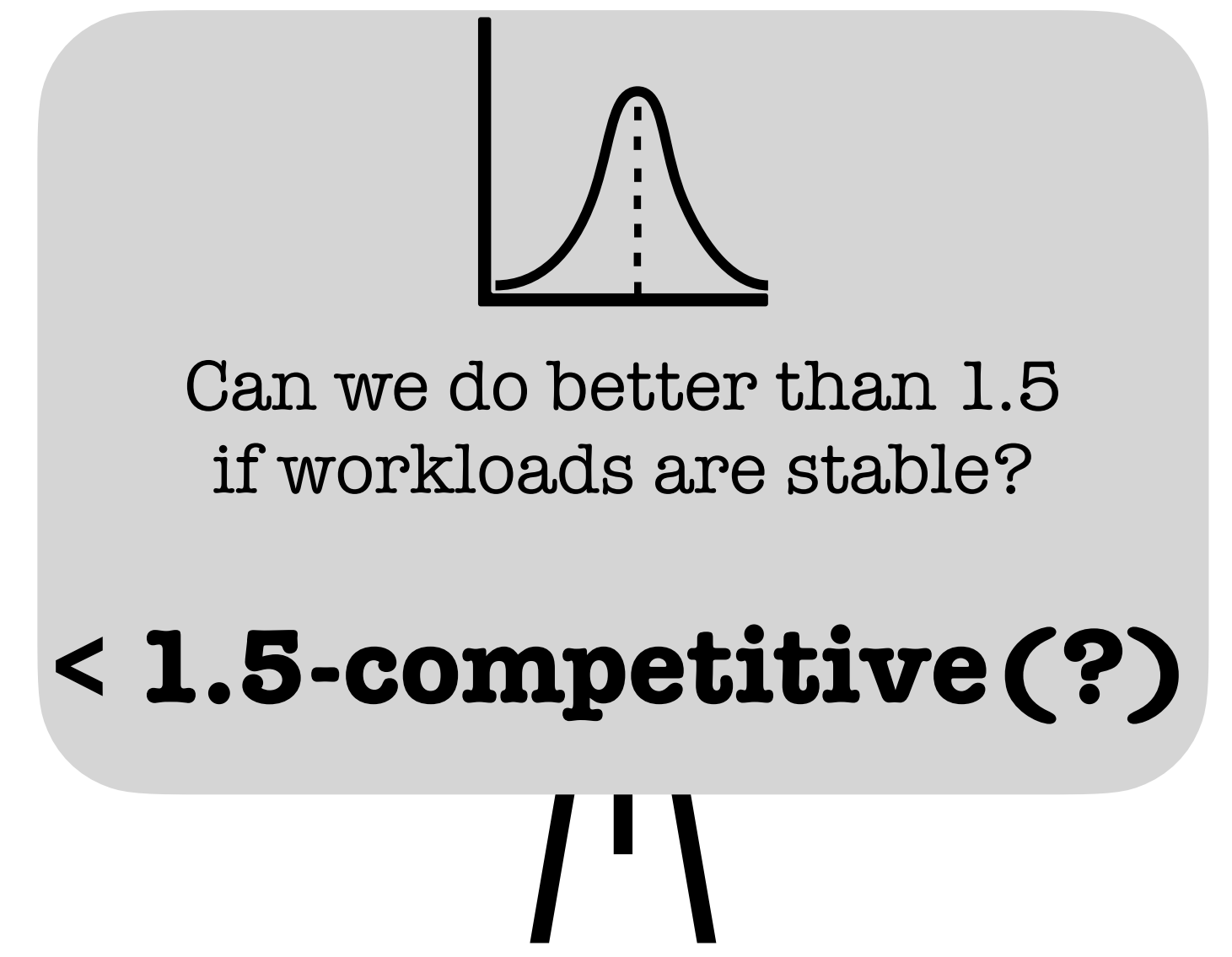
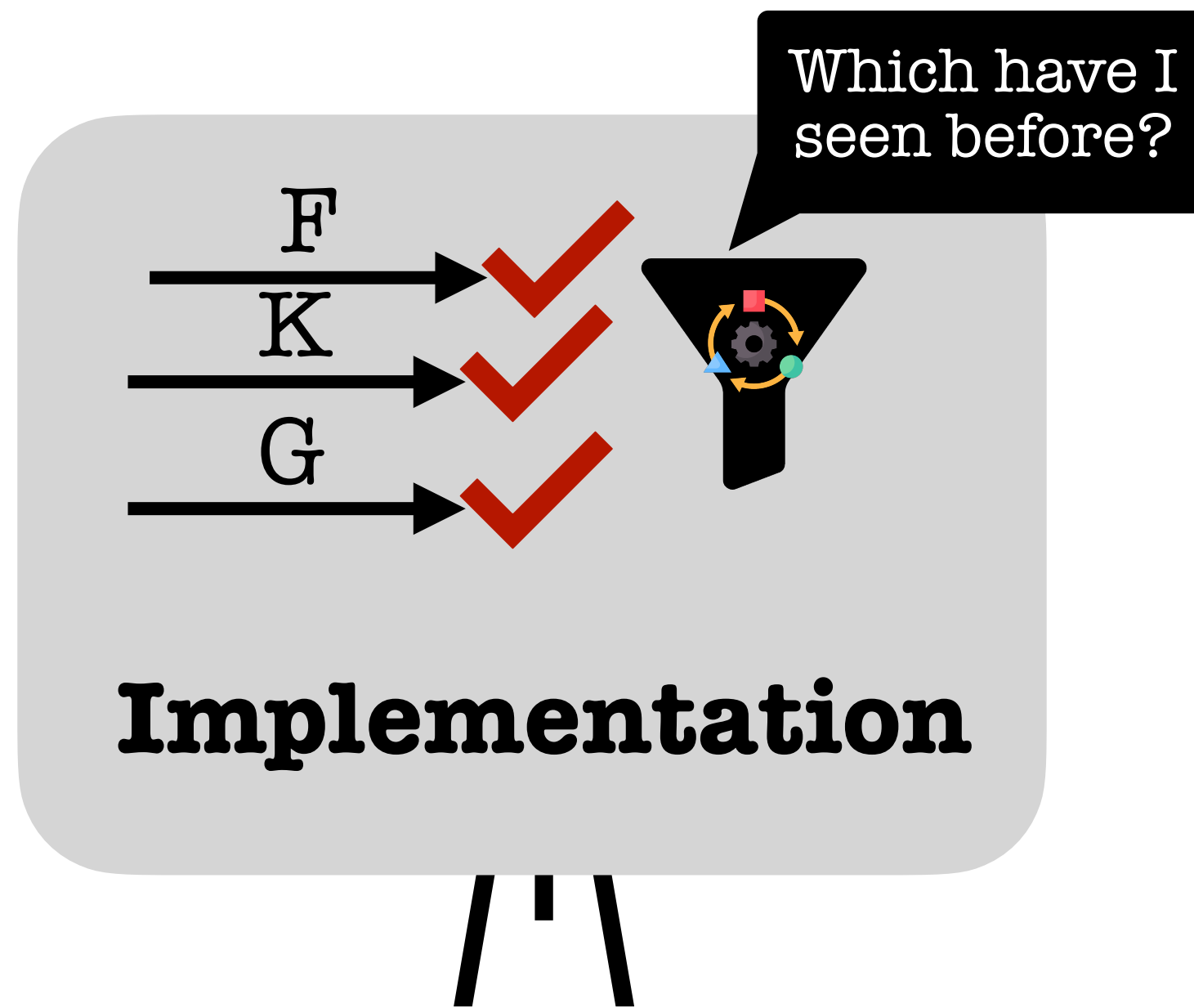
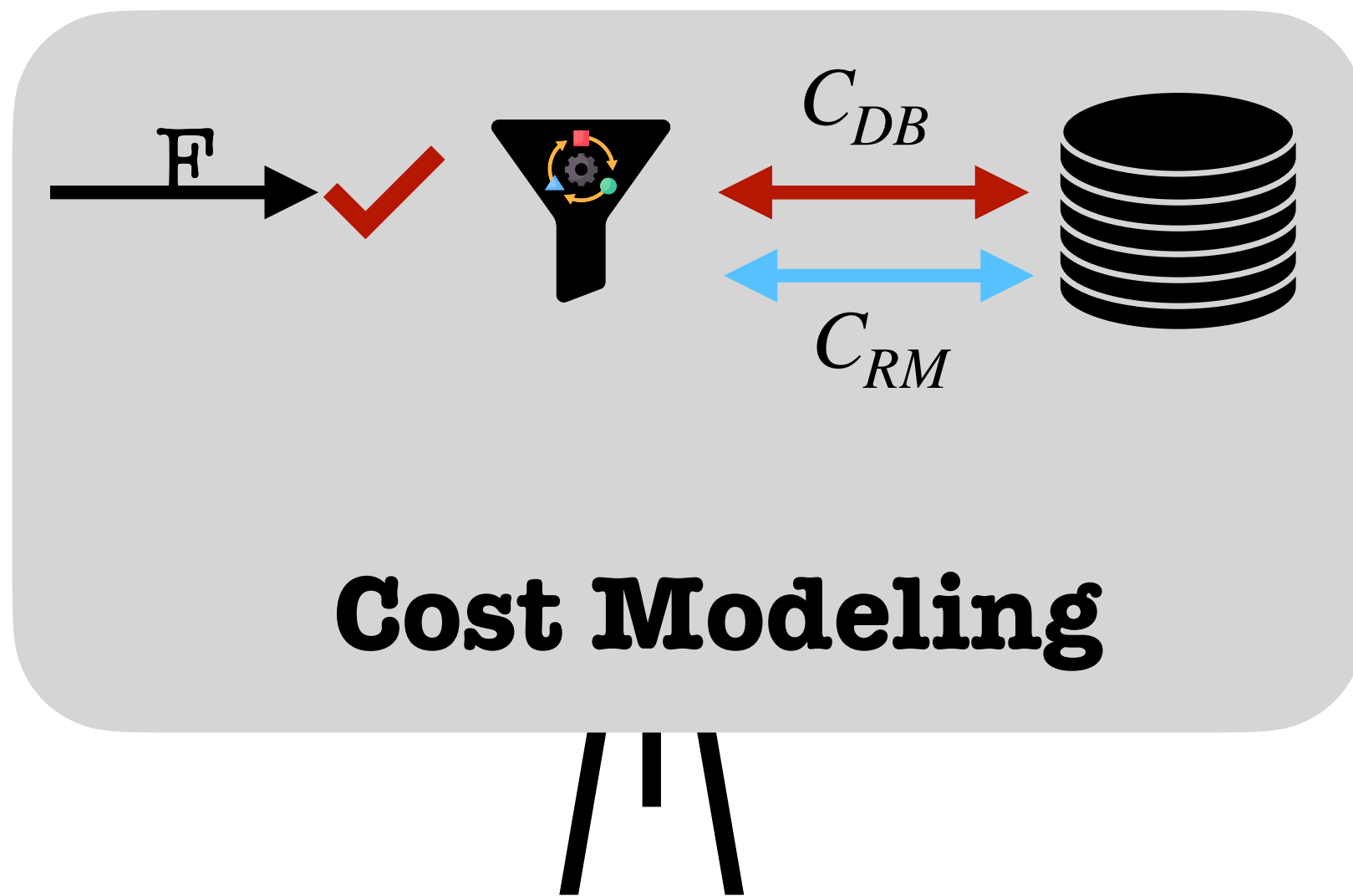
Competitive ratio: 2

Non-adaptive Filter
(never adapt)

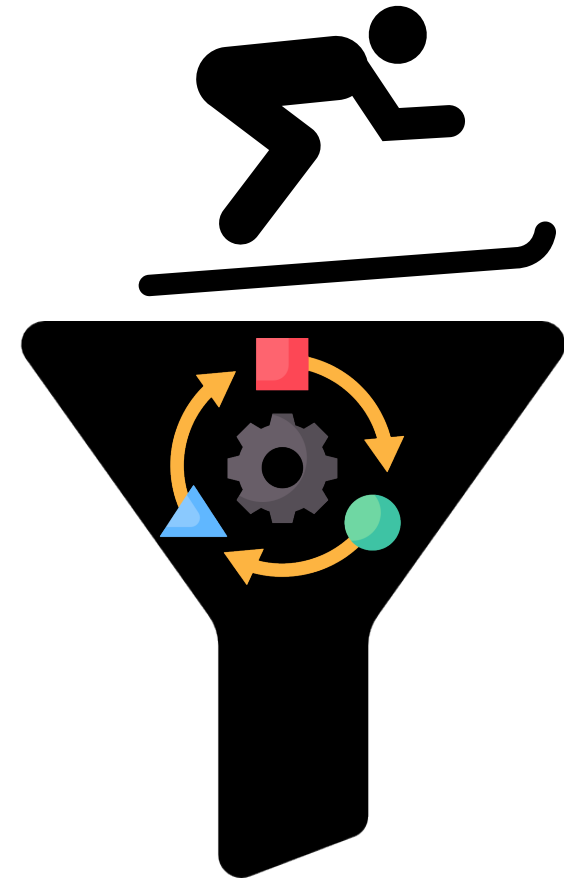


Competitive ratio: n
(number of queries)

!! Poster session !!

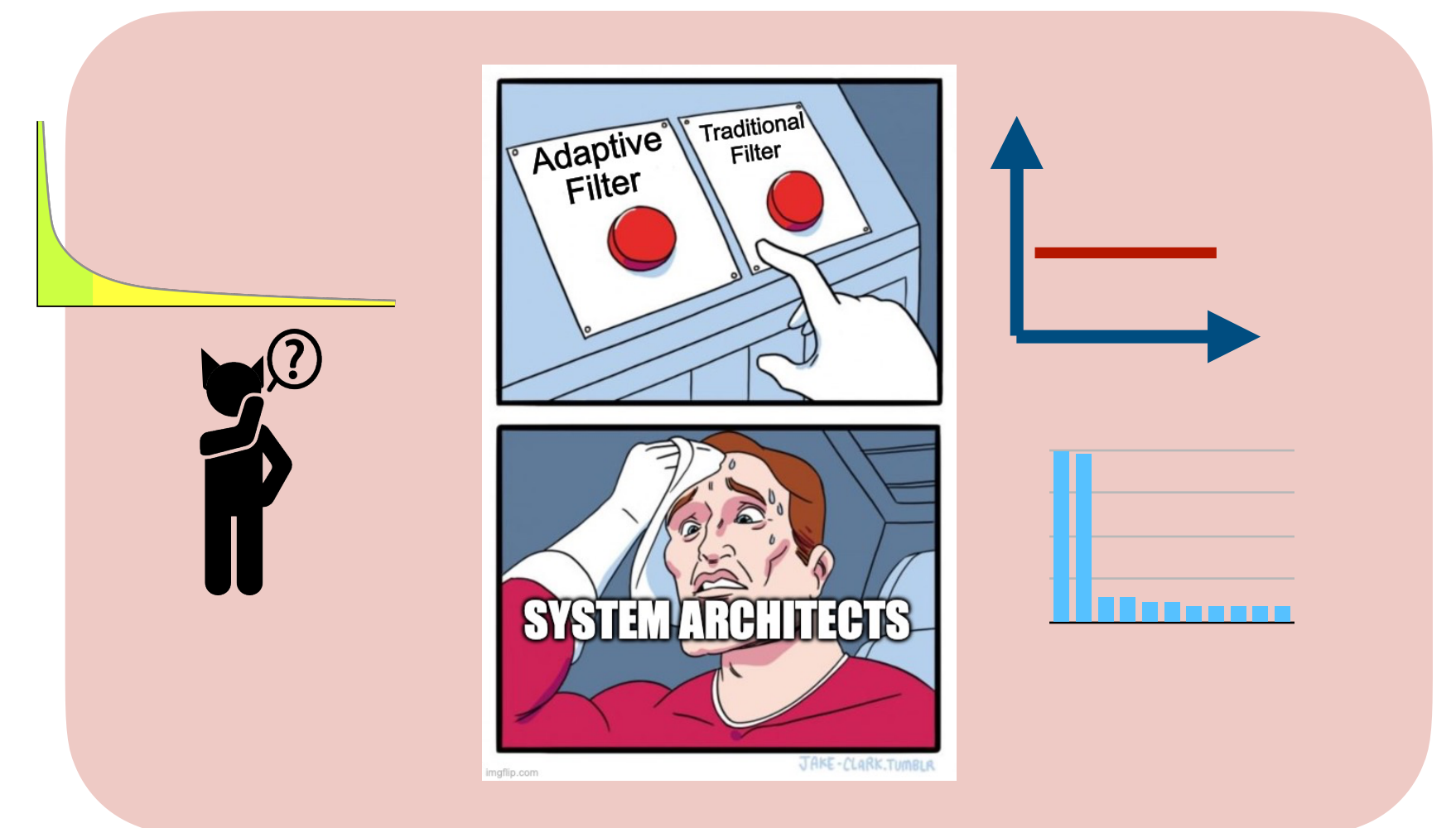
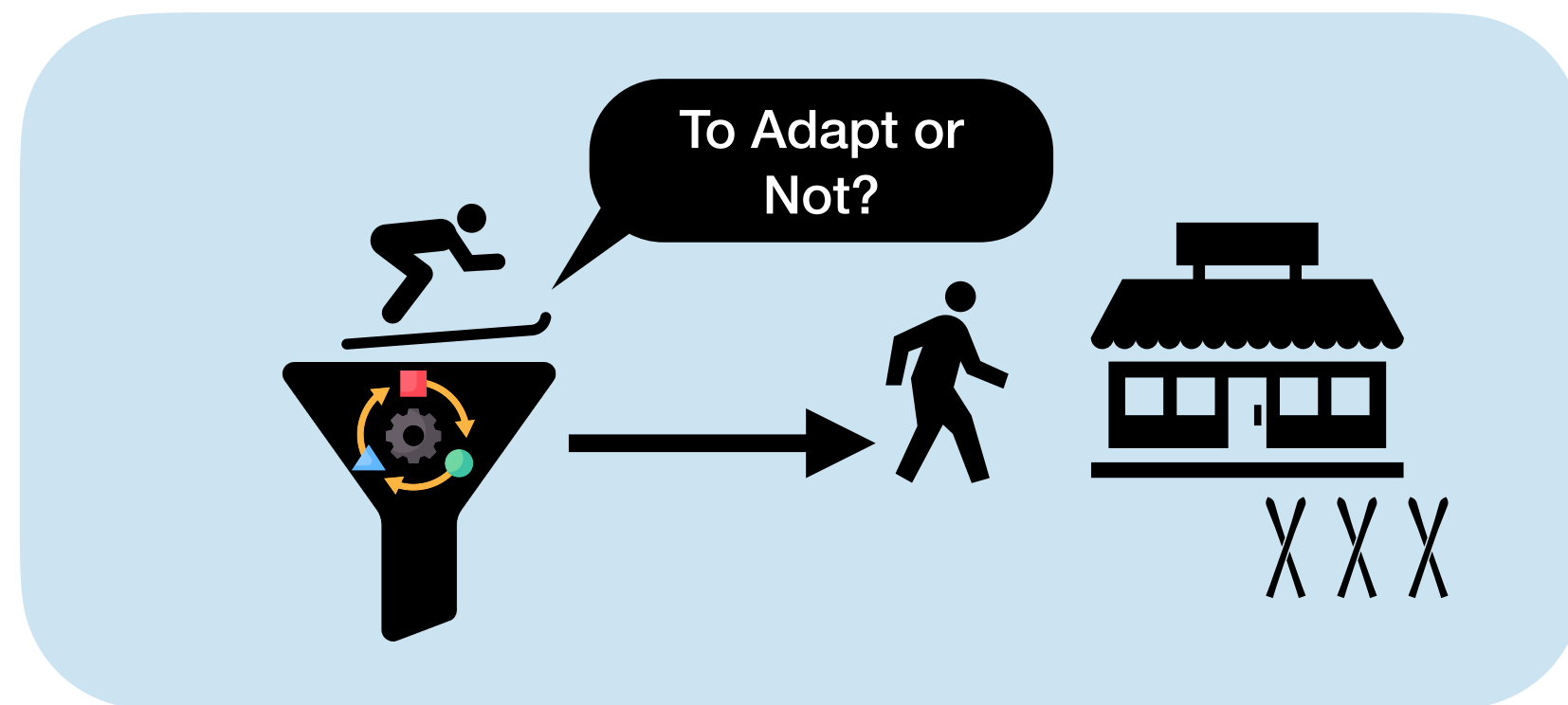


Takeaways



An **online-adaptive** filter that provides:

- ✓ **1.5-competitive** performance
- ✓ **No workload analysis** required
- ✓ **Minimal** memory/CPU overhead



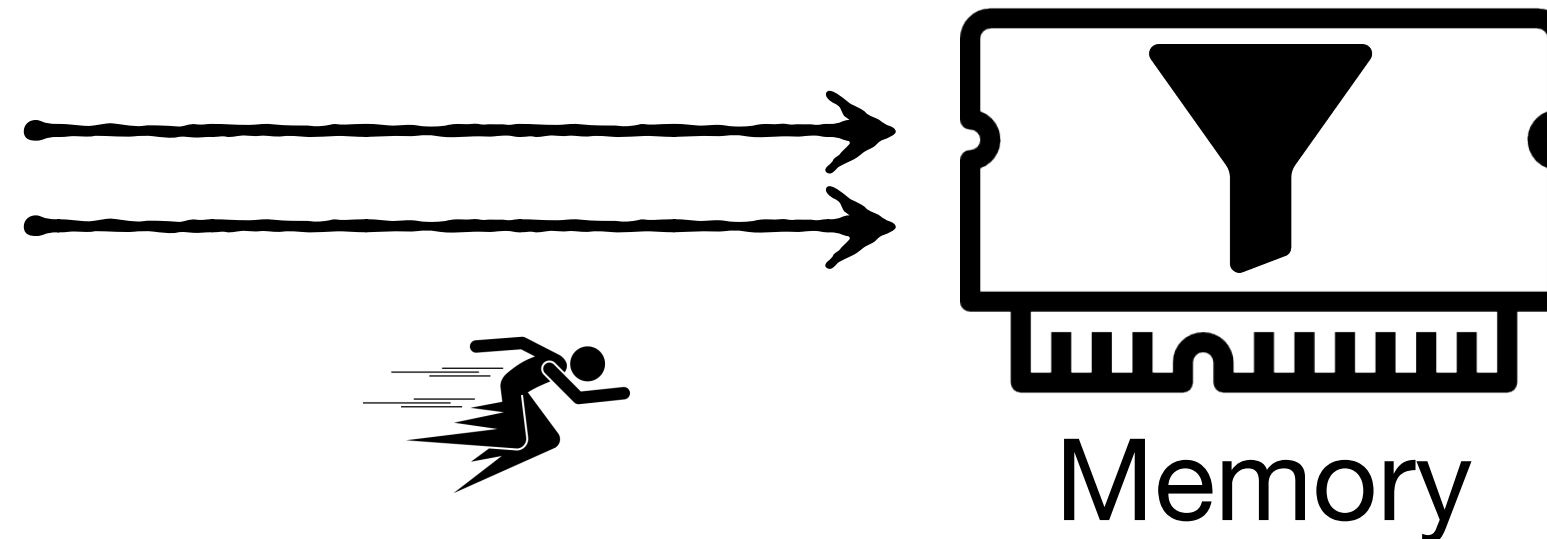
Evaluation Results (Backup)

All **evaluations** measure **end-2-end throughput**

Workload

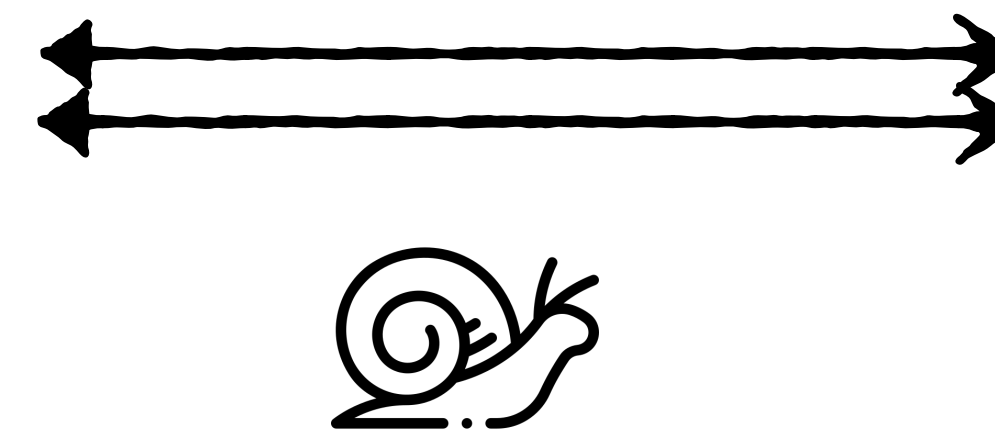


**Filter in
main-memory**



Memory

**Disk-based
database**



Uniform Random
Normal
LogNormal
Zipf (0.99)
Adversarial

Non-Adaptive (Counting Quotient Filter)

Adaptive (Adaptive QuotientFilter)

SkiQF (Ours)

Throughput across distributions (Higher is better)

